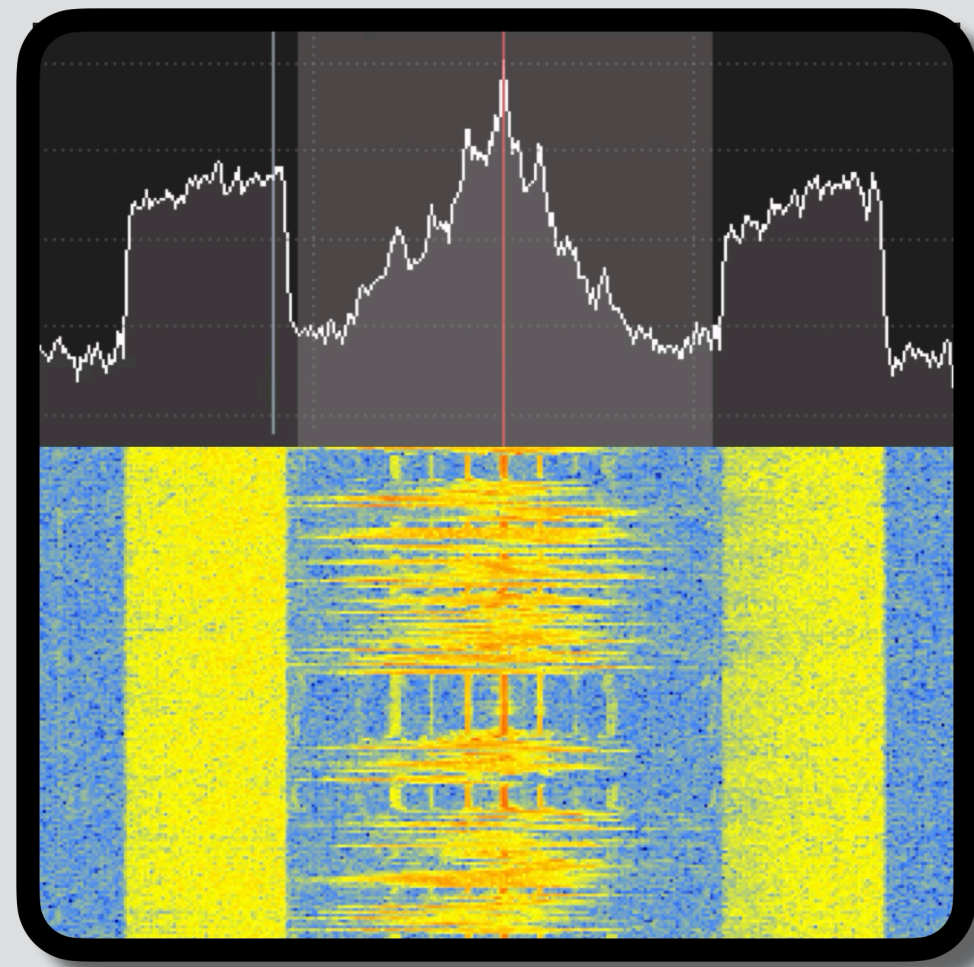
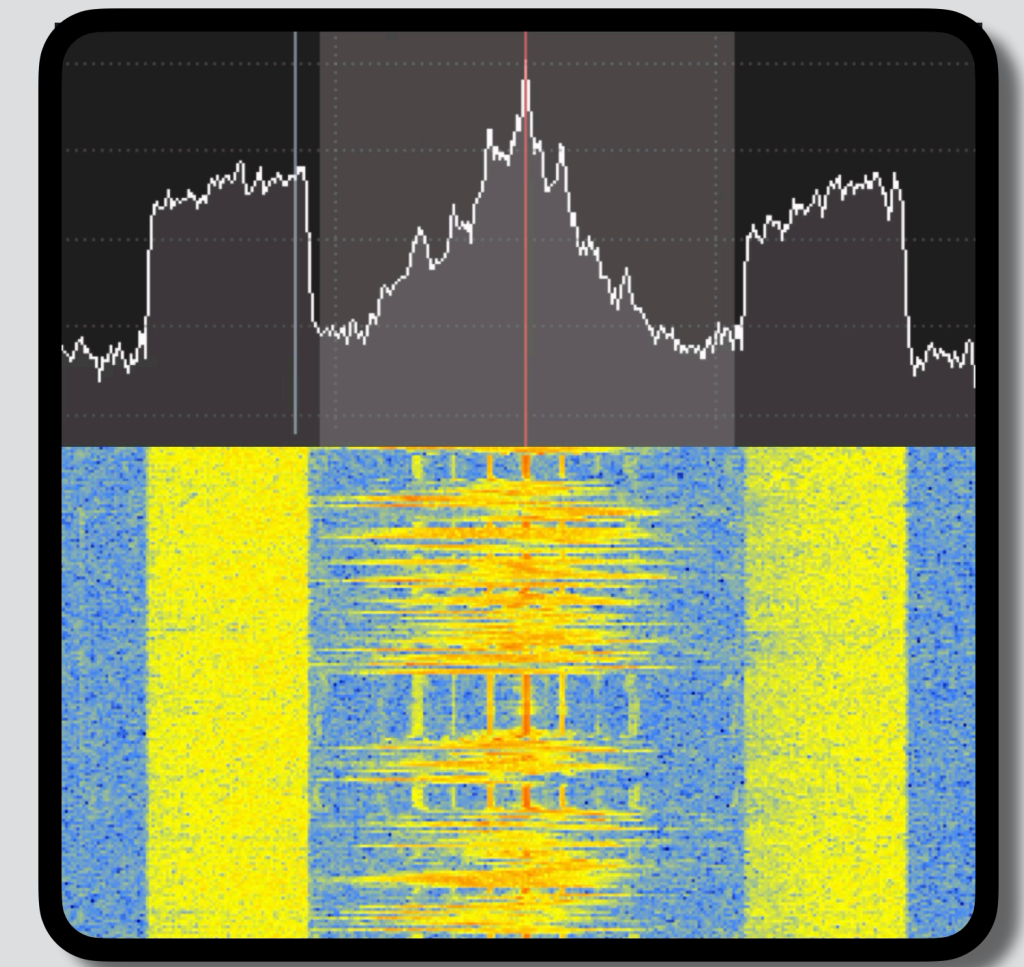




EE121

& Analog



Introduction to Digital Communication Systems

Lecture 12B

Miki Lustig

Electrical Engineering and Computer Sciences, UC Berkeley

Today

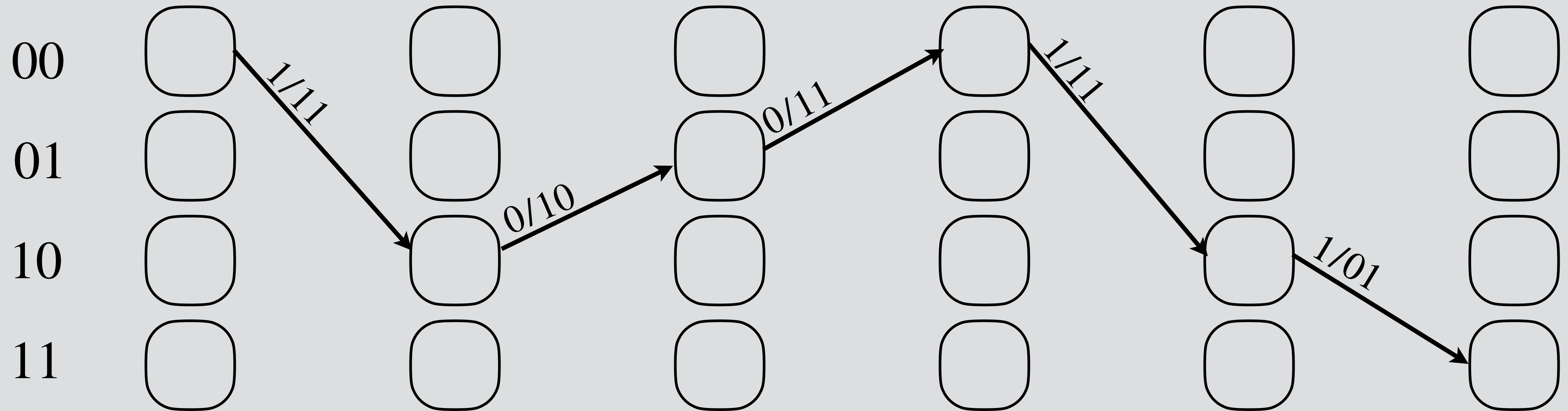
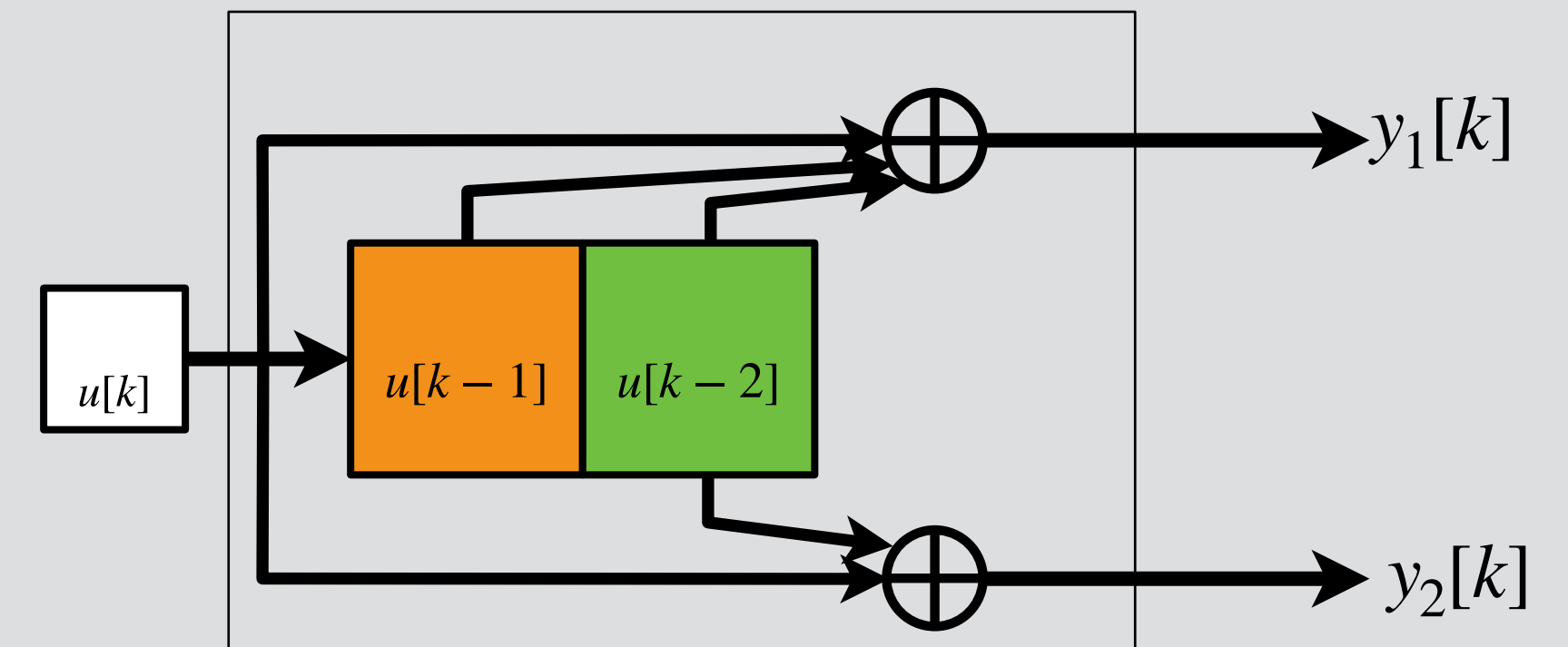
- Last time:
 - Convolutional Codes
 - Viterbi Decoding
- Today: Andrews notes (Bcourses)
 - Soft Decoding
 - Puncturing
 - Block Codes - Hamming

Trellis diagram

Data: $u = 10011$

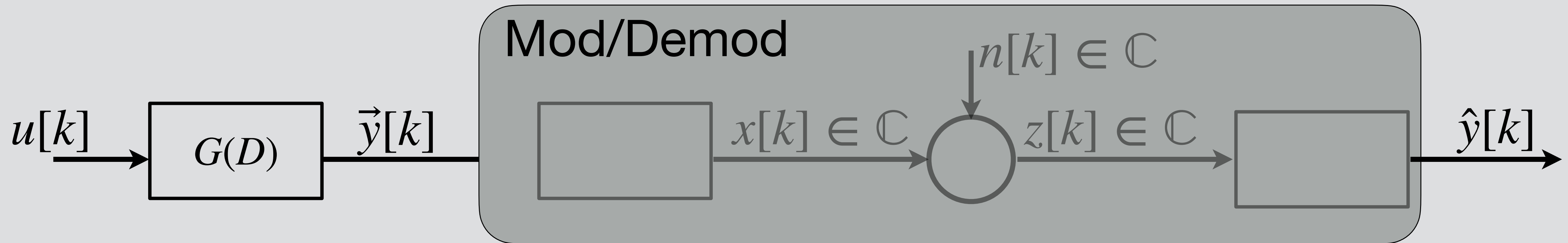
Sent: $y = 1110111101$

Tx bits set a path. Recover path, recover bits



Towards Decoding

Consider an AWGN channel with the following encoder / decoder:

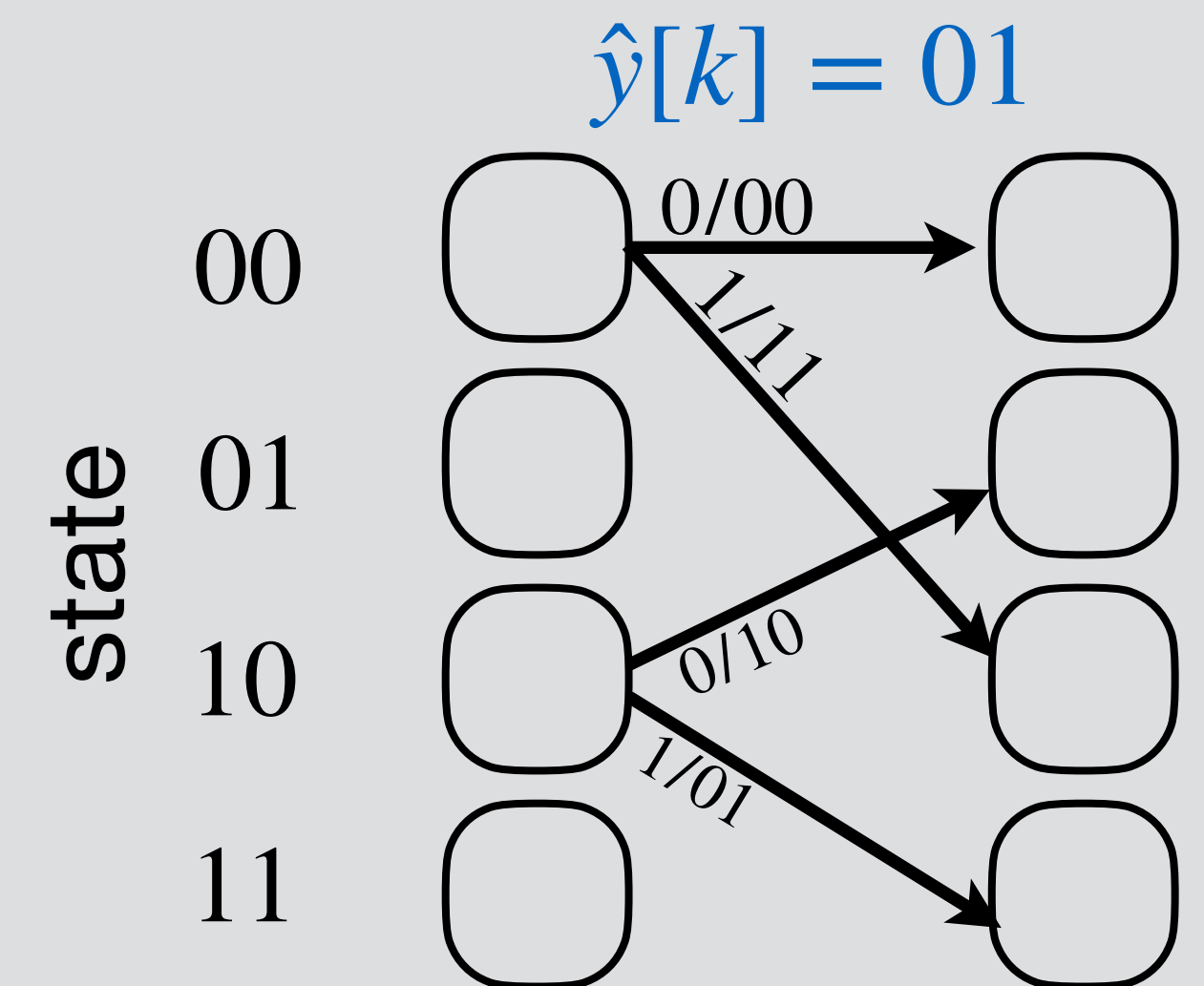


Define *branch metric* $\lambda_k(s[k] \rightarrow s[k + 1])$

Cost of moving from one state to another:

$$\lambda_k(s[k] \rightarrow s[k + 1]) = \sum_i \hat{y}_i[k] \oplus y_i[k]$$

Without noise, cost should be = 0!

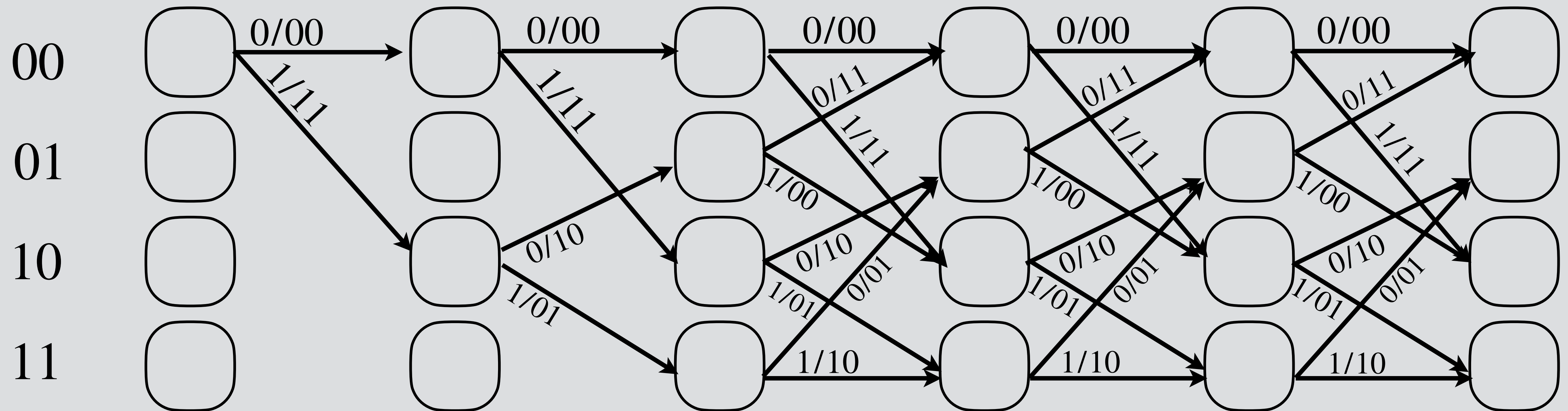
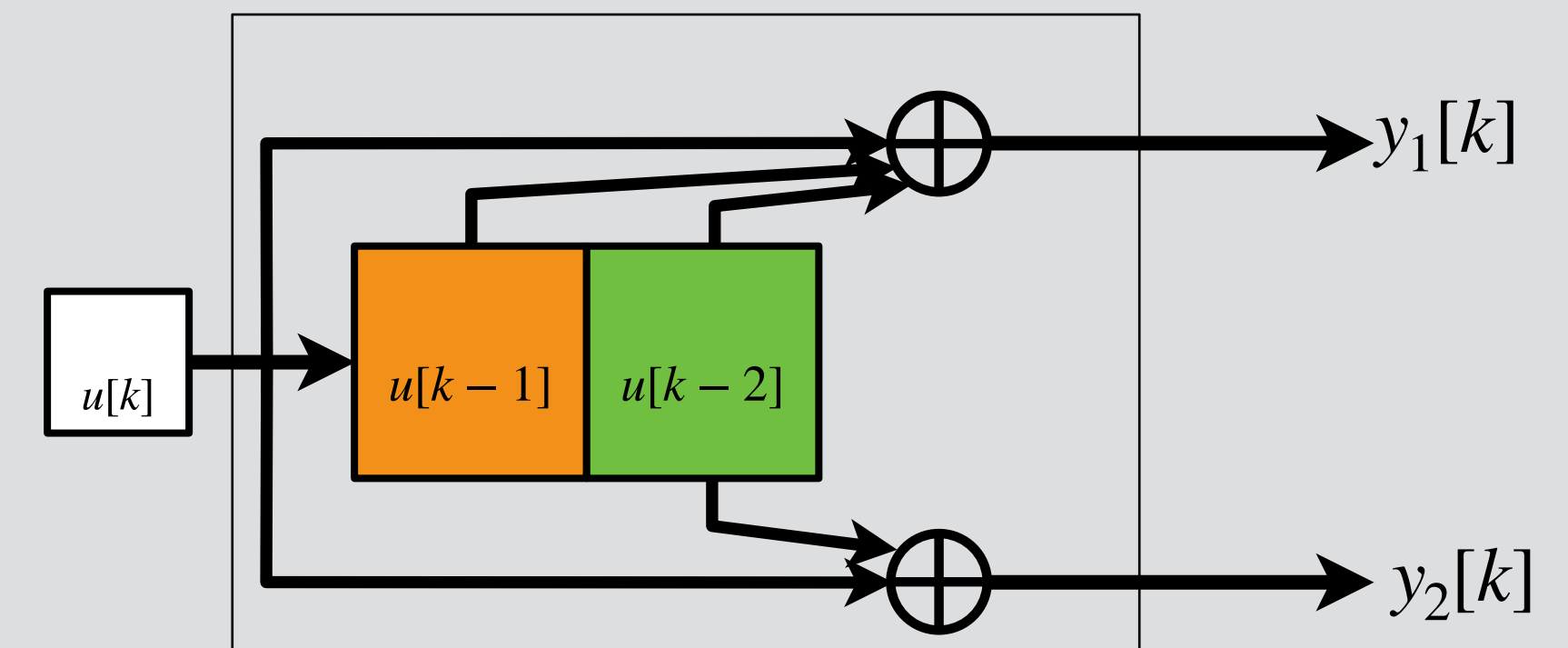


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

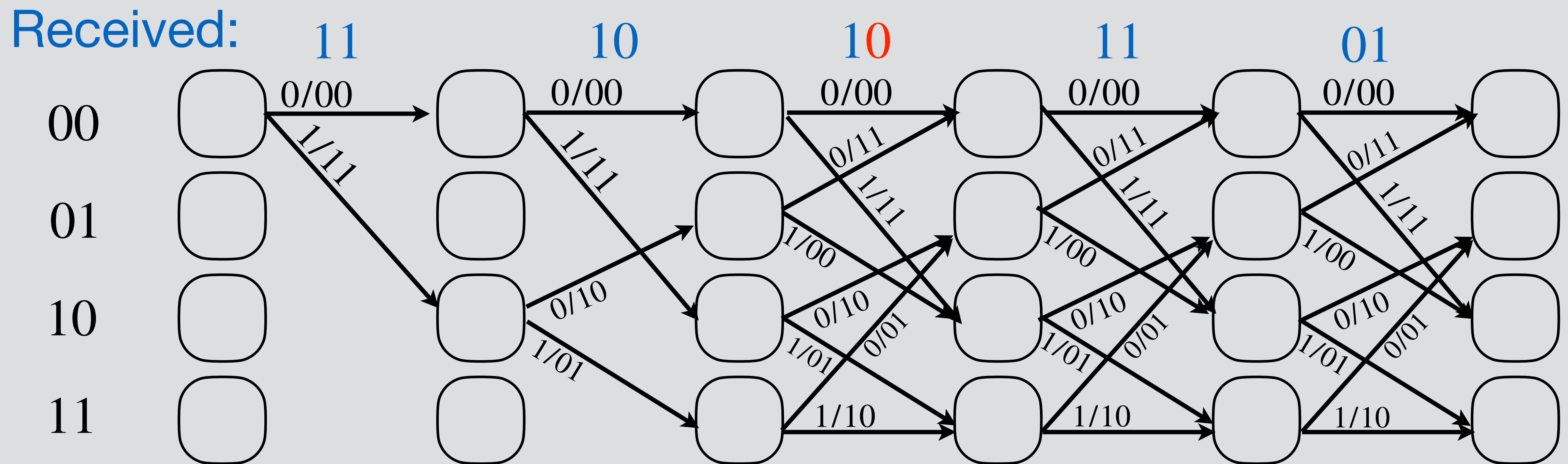
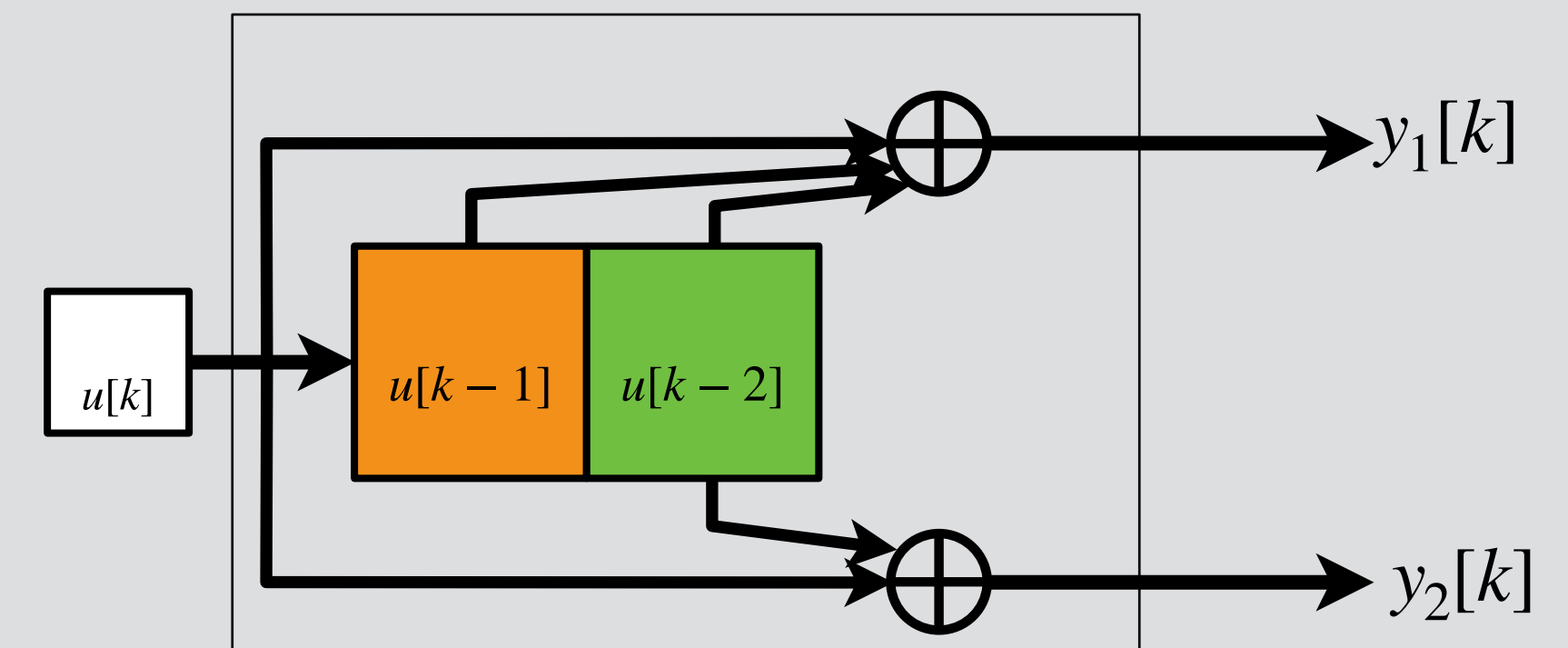


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

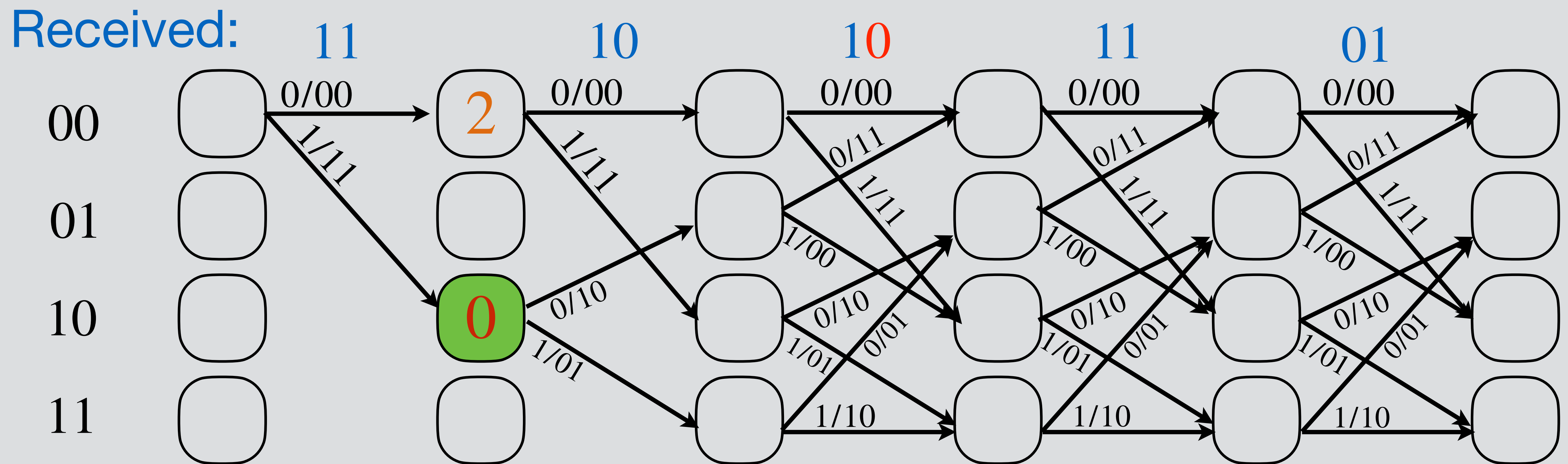
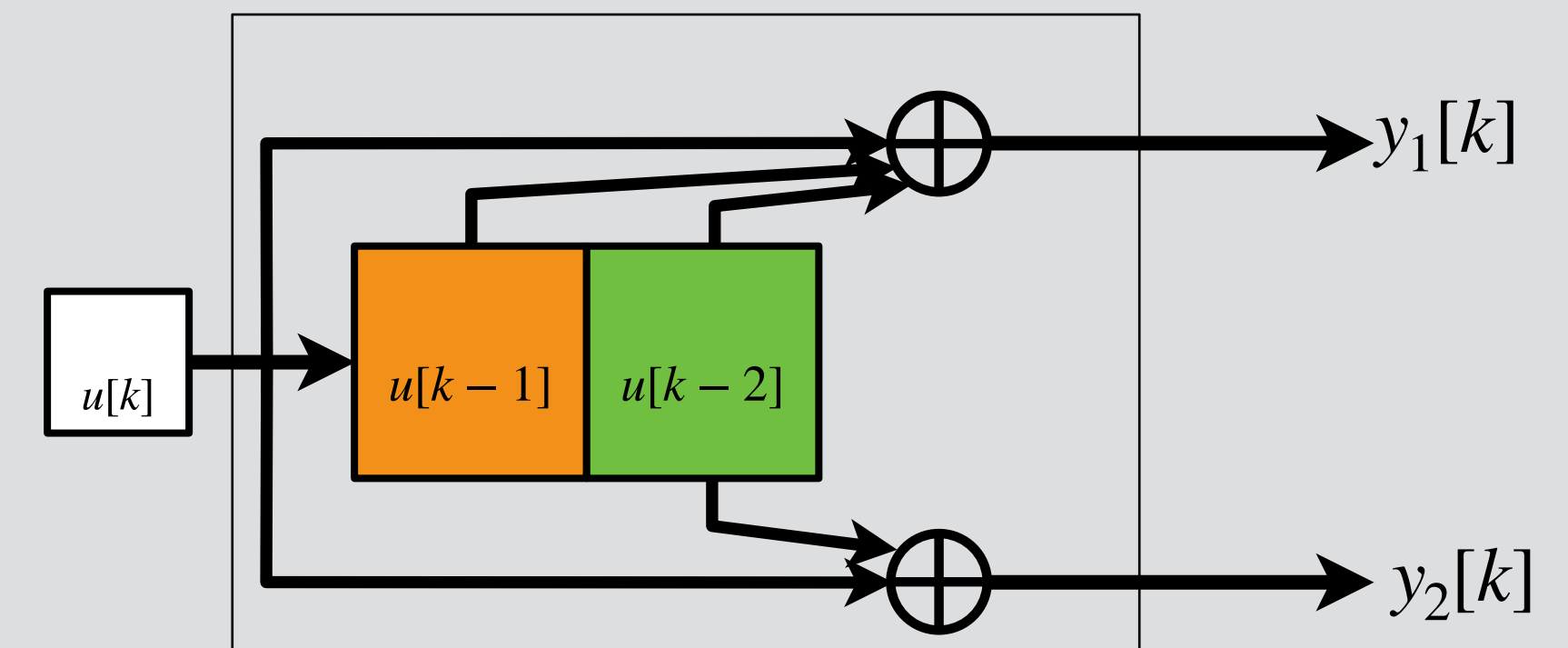


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

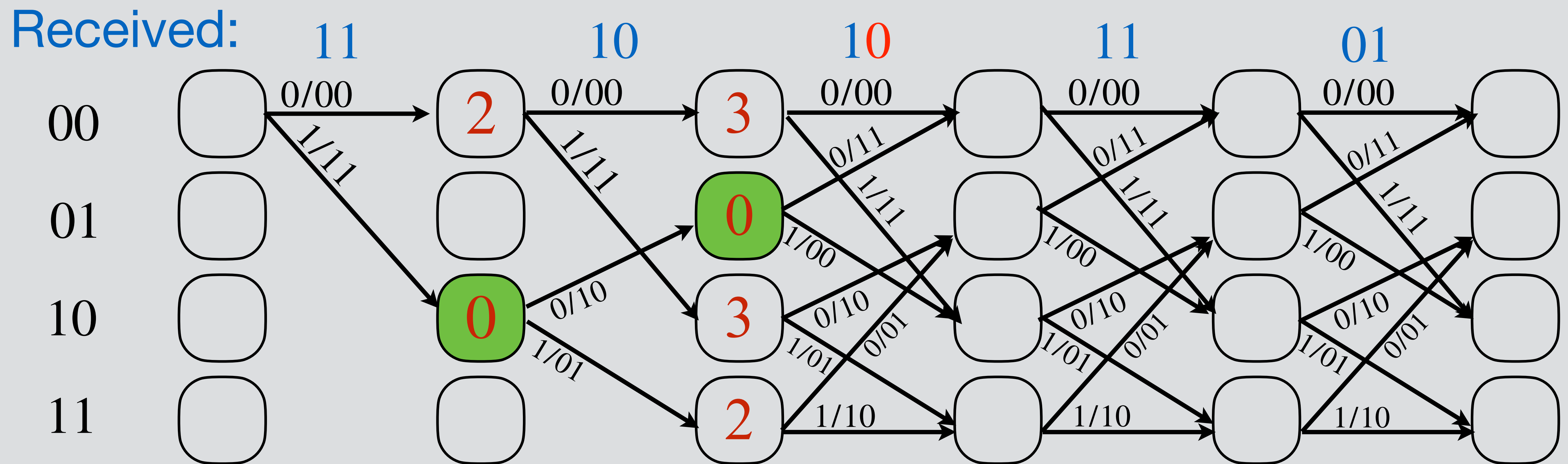
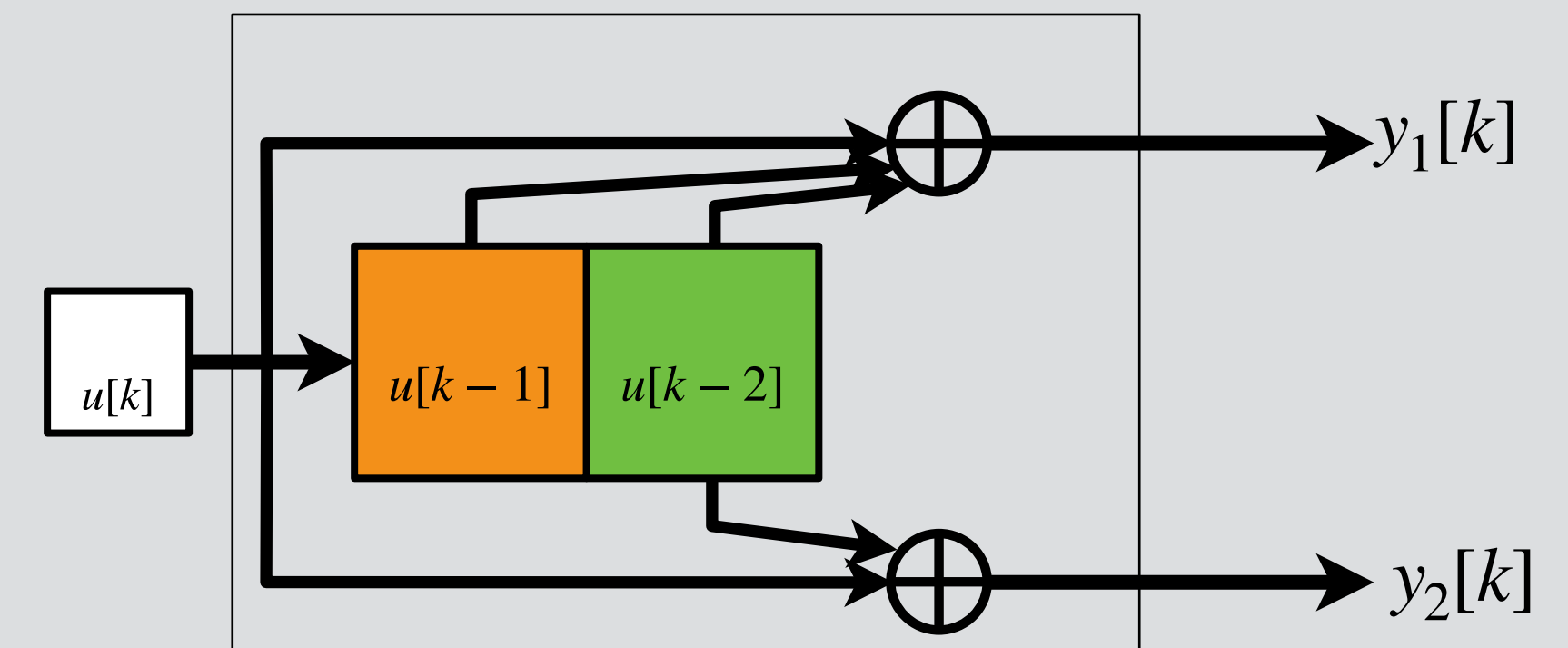


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

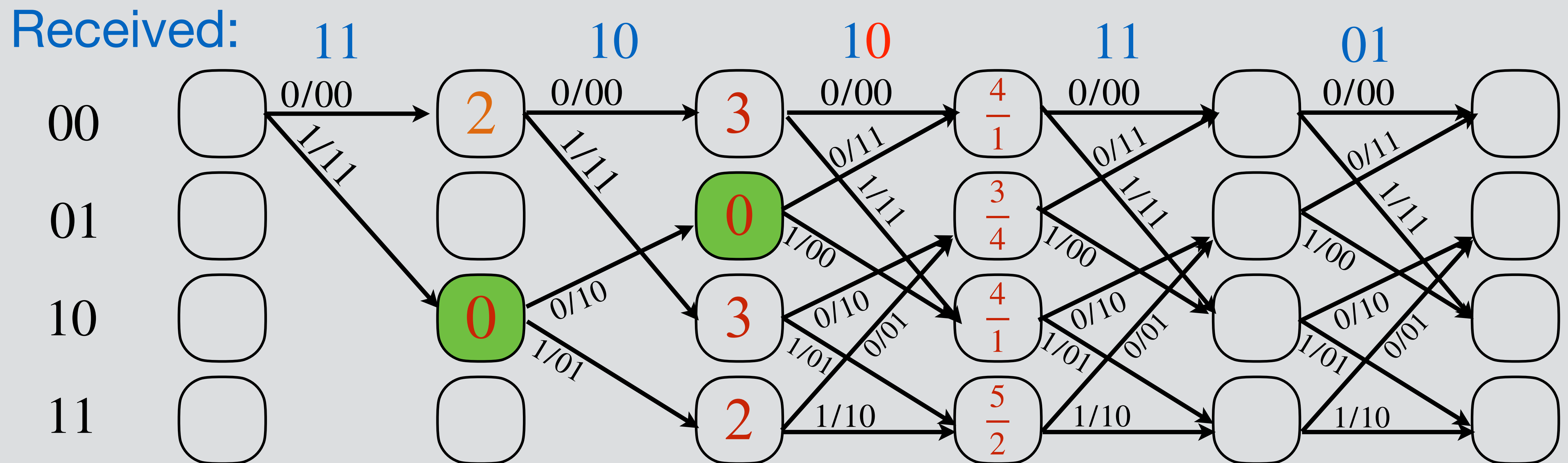
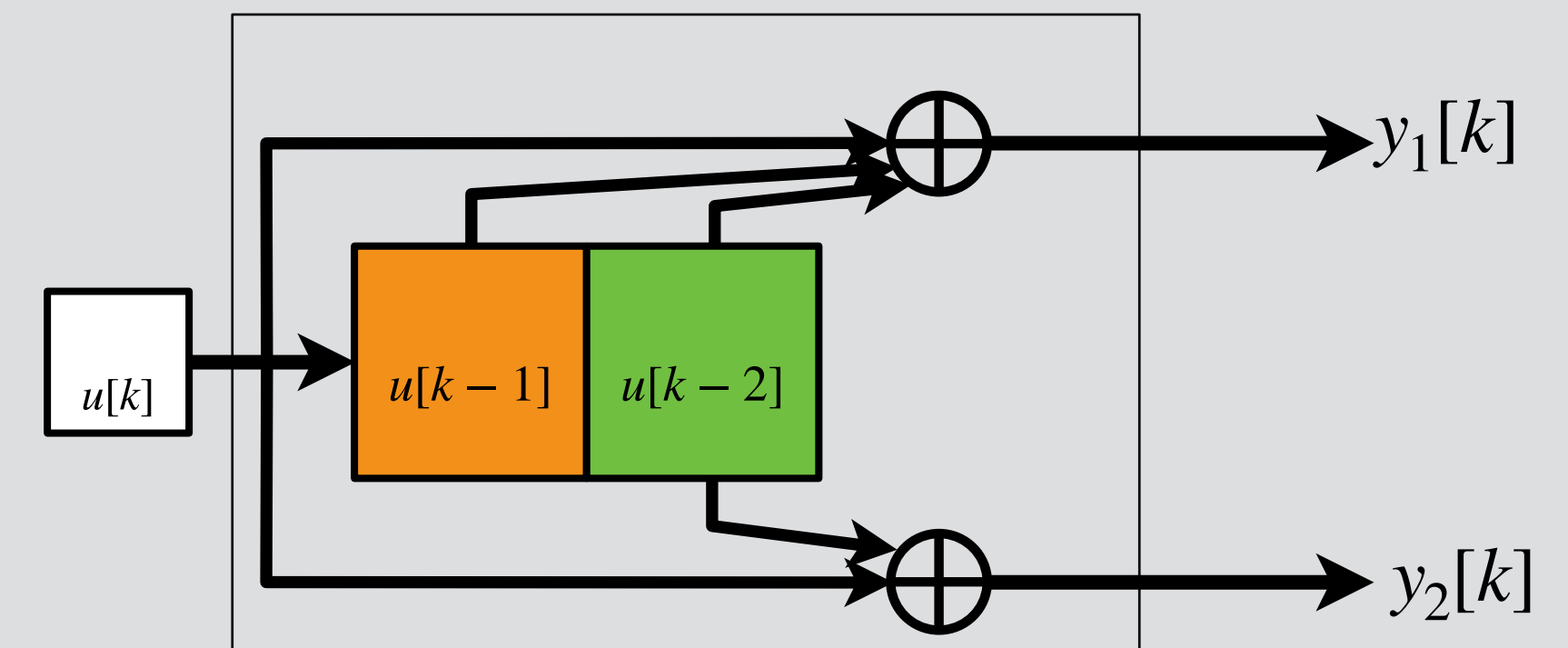


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



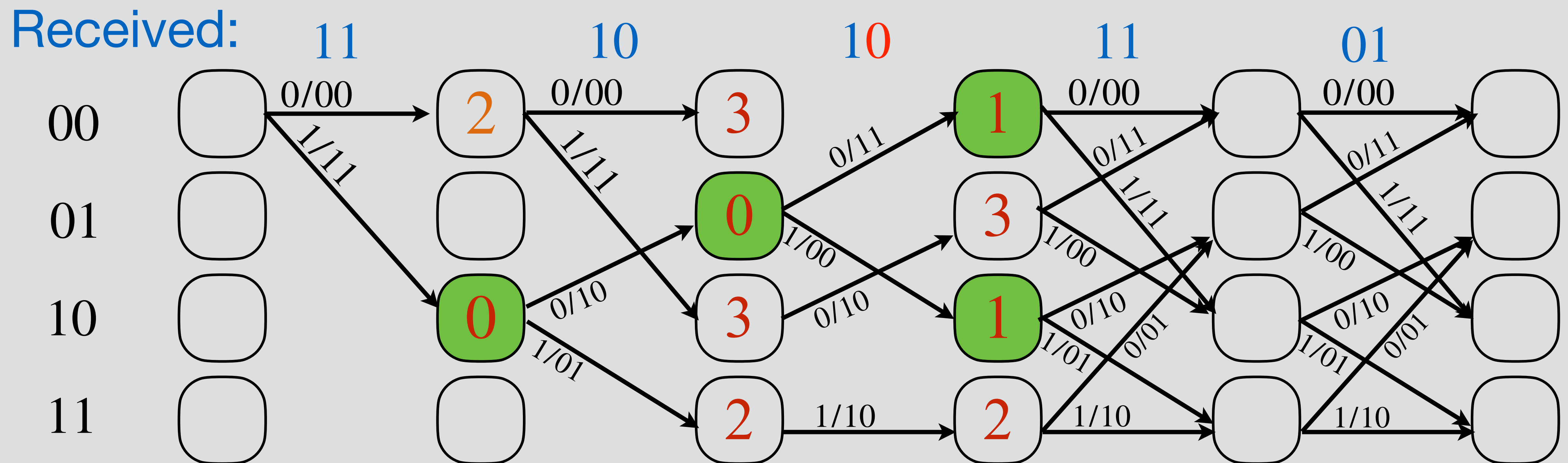
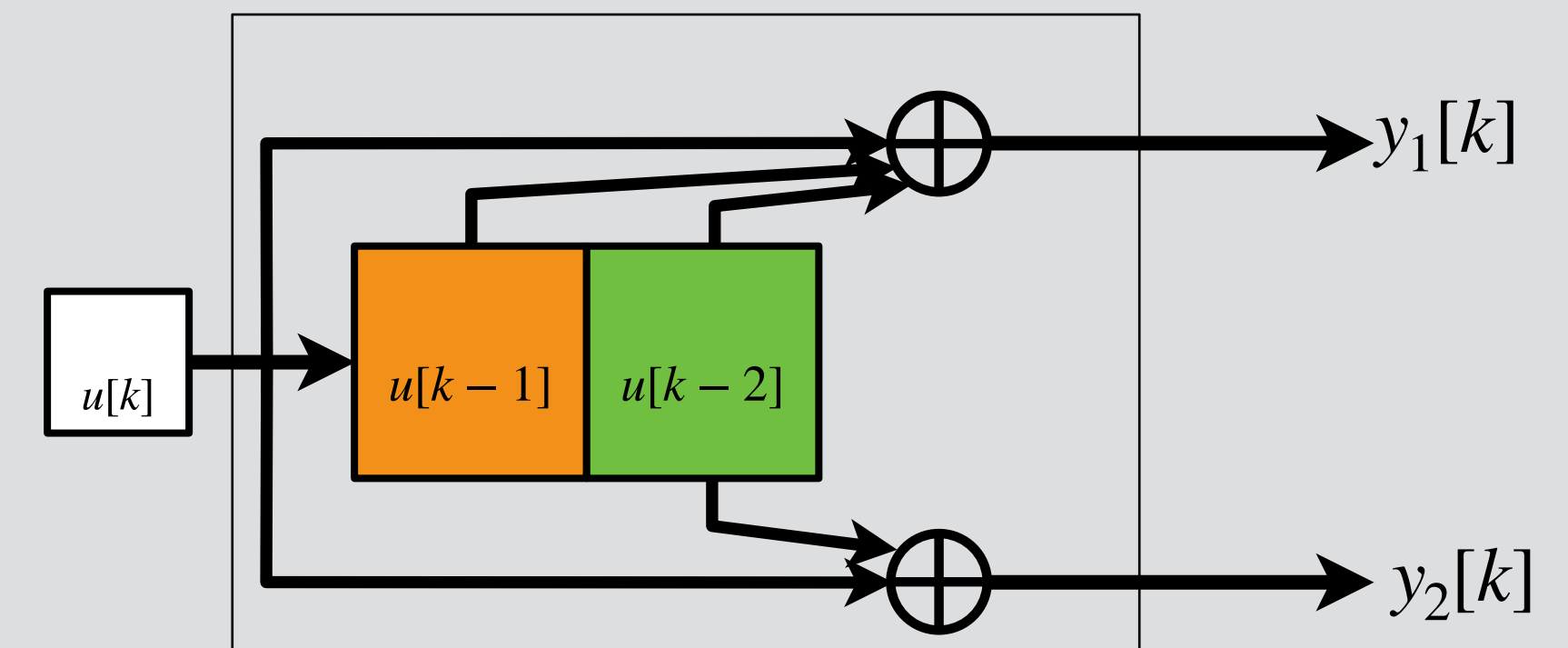
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



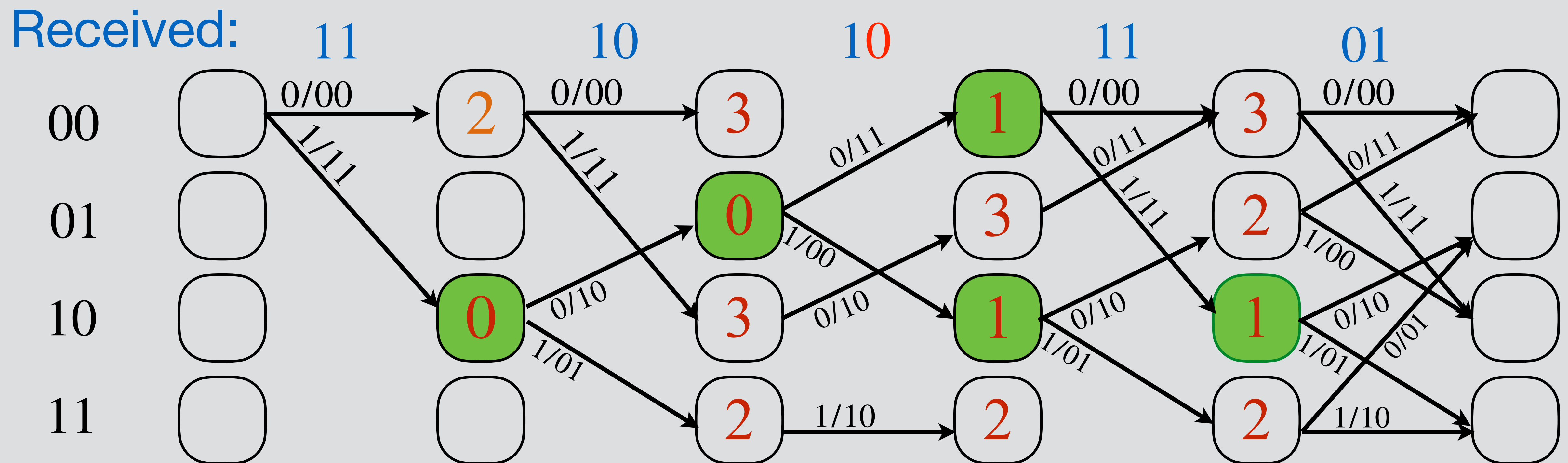
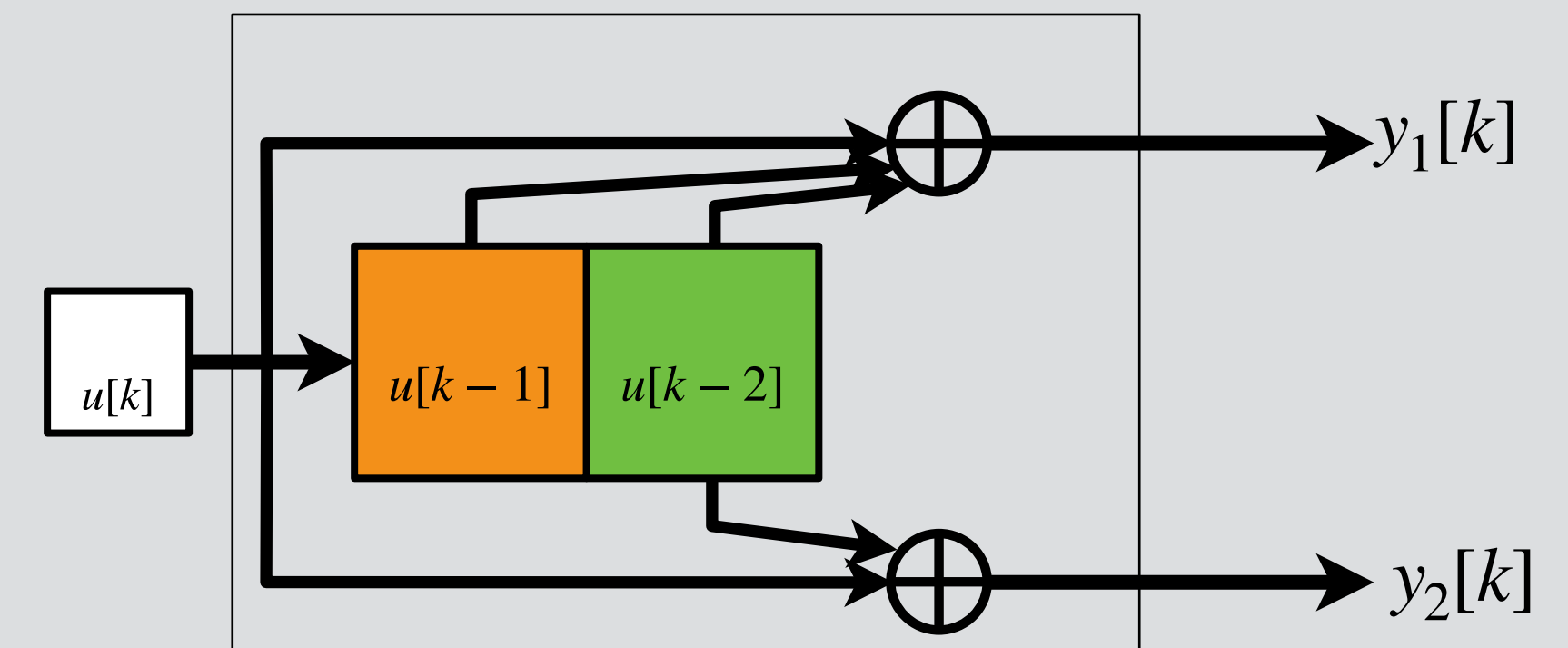
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



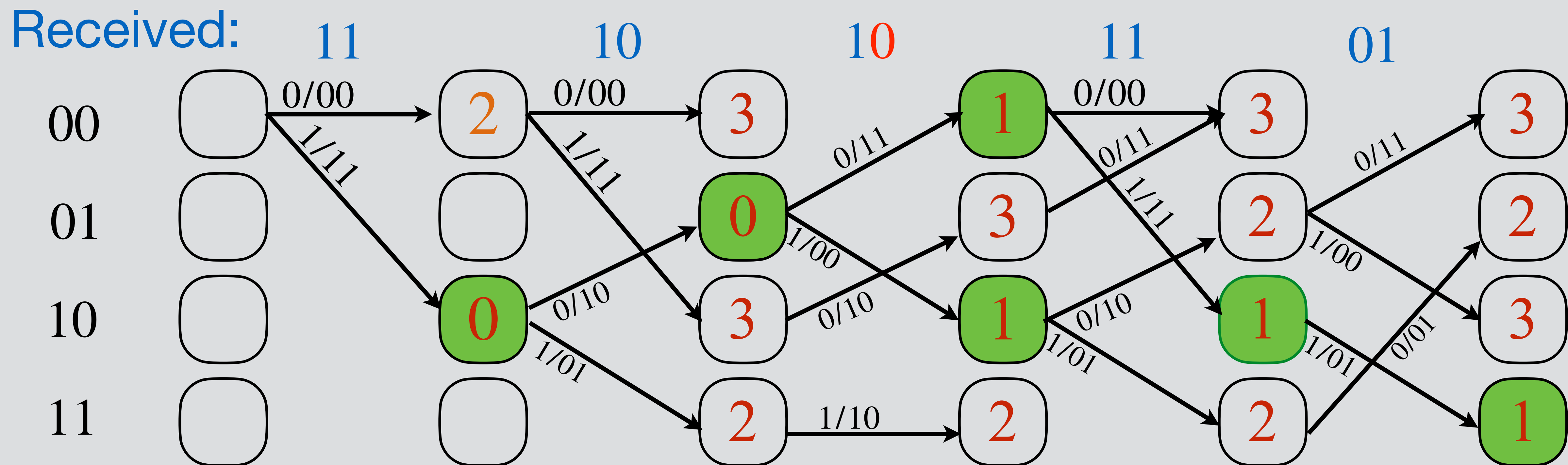
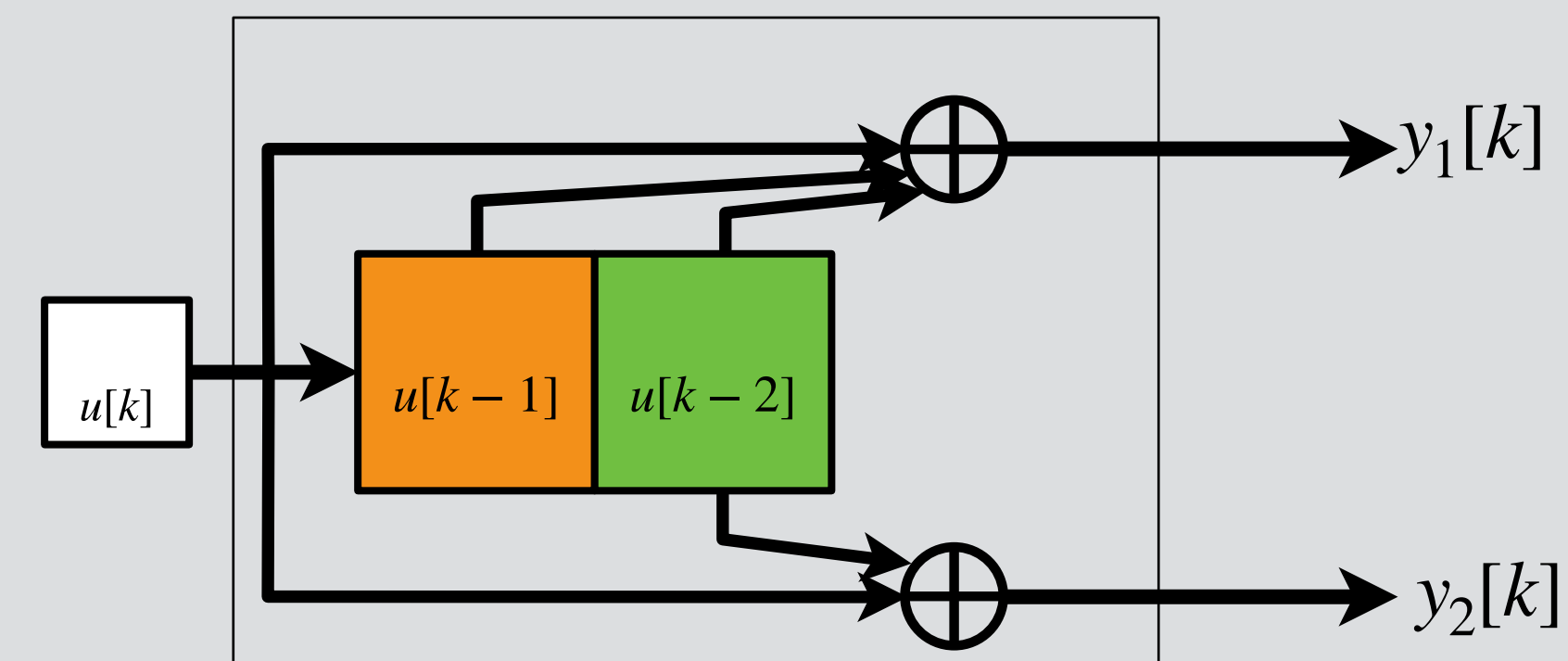
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



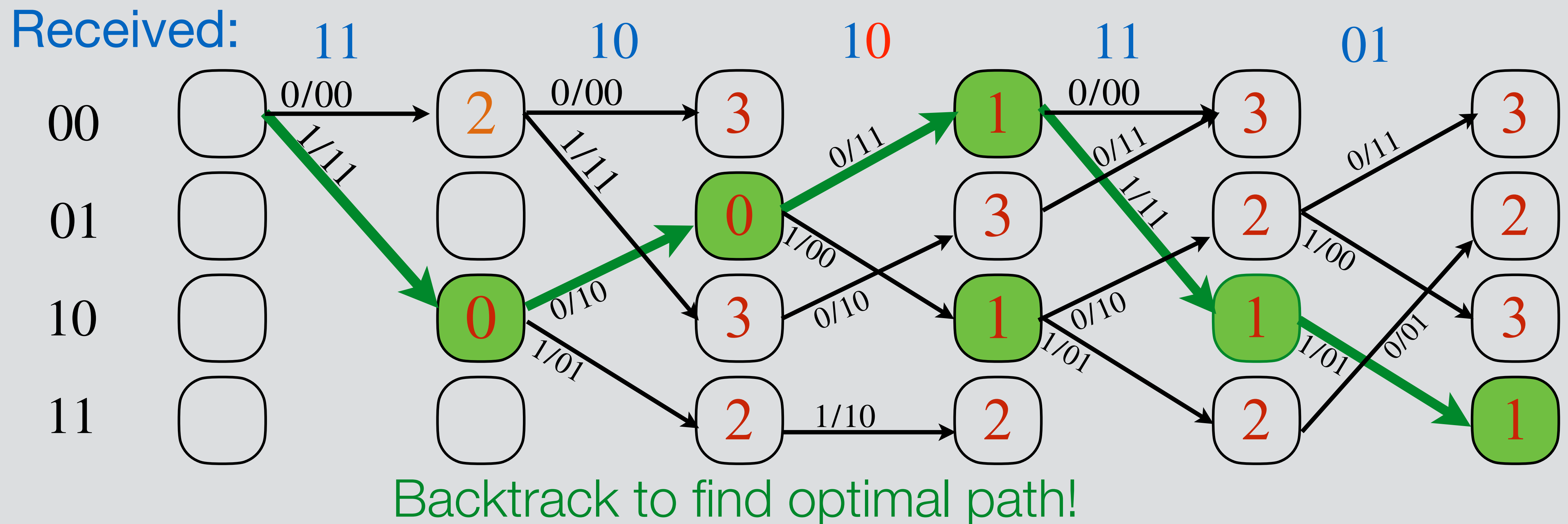
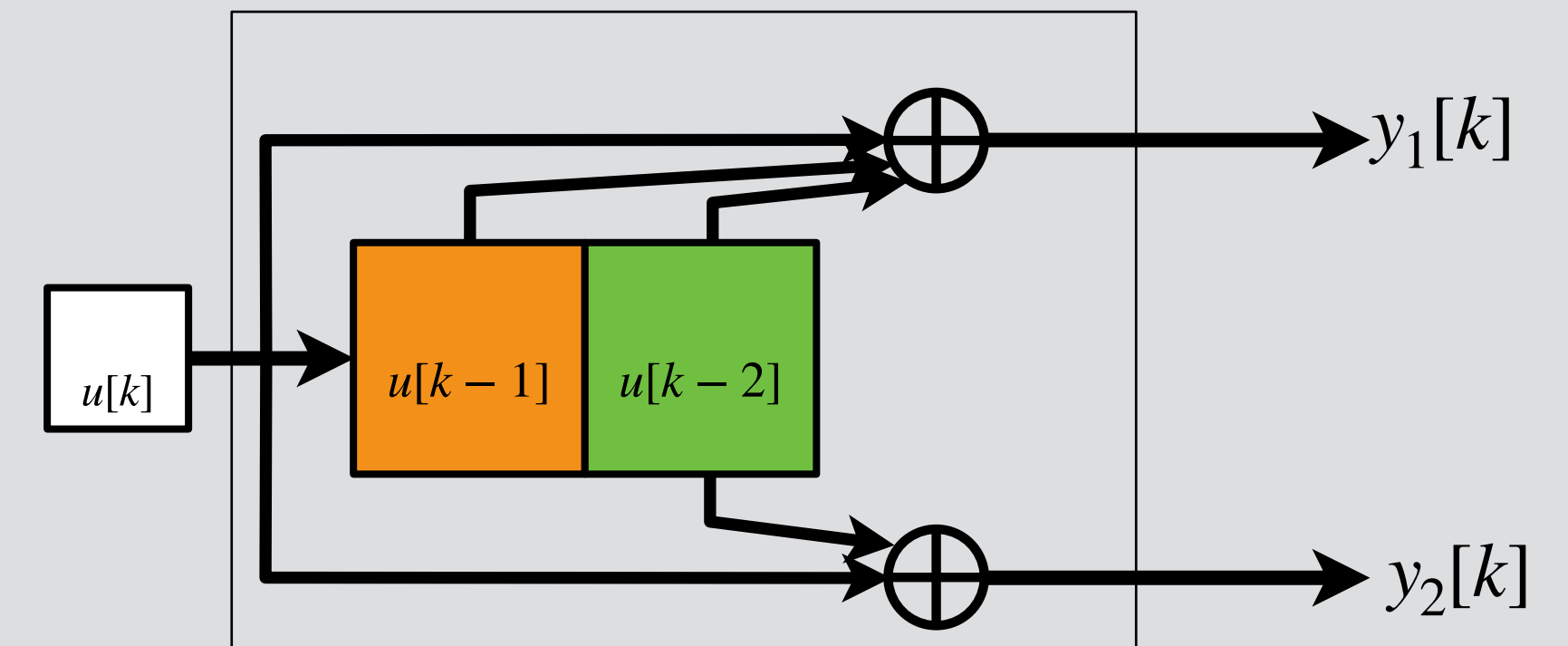
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 1110101101$

Exponential number of paths

Let's compute each branch metric (cost)

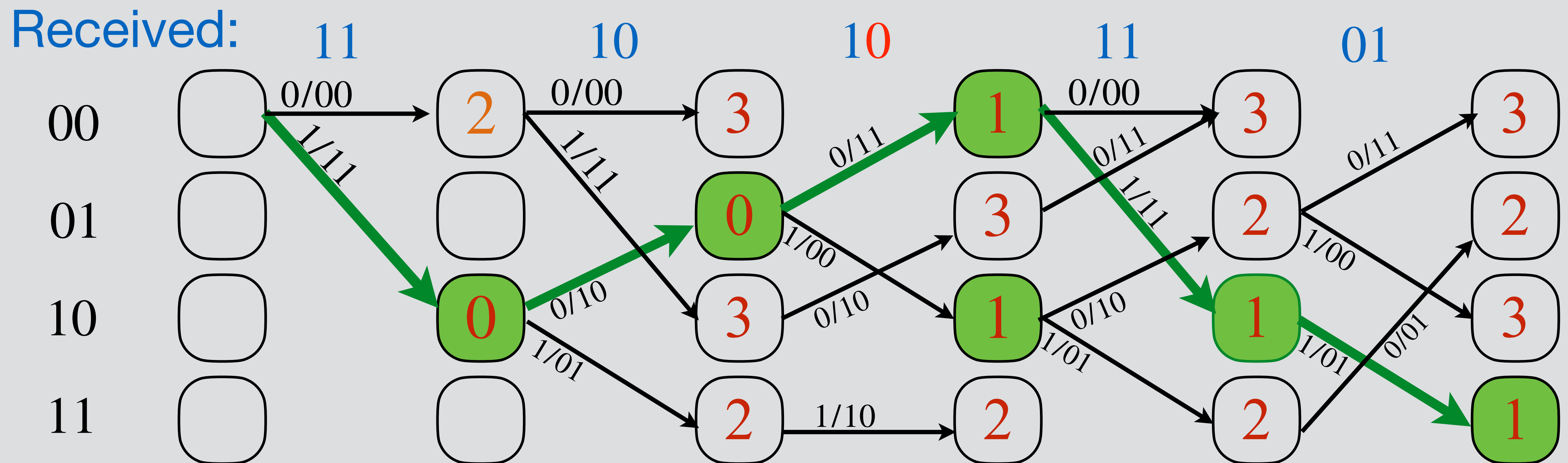
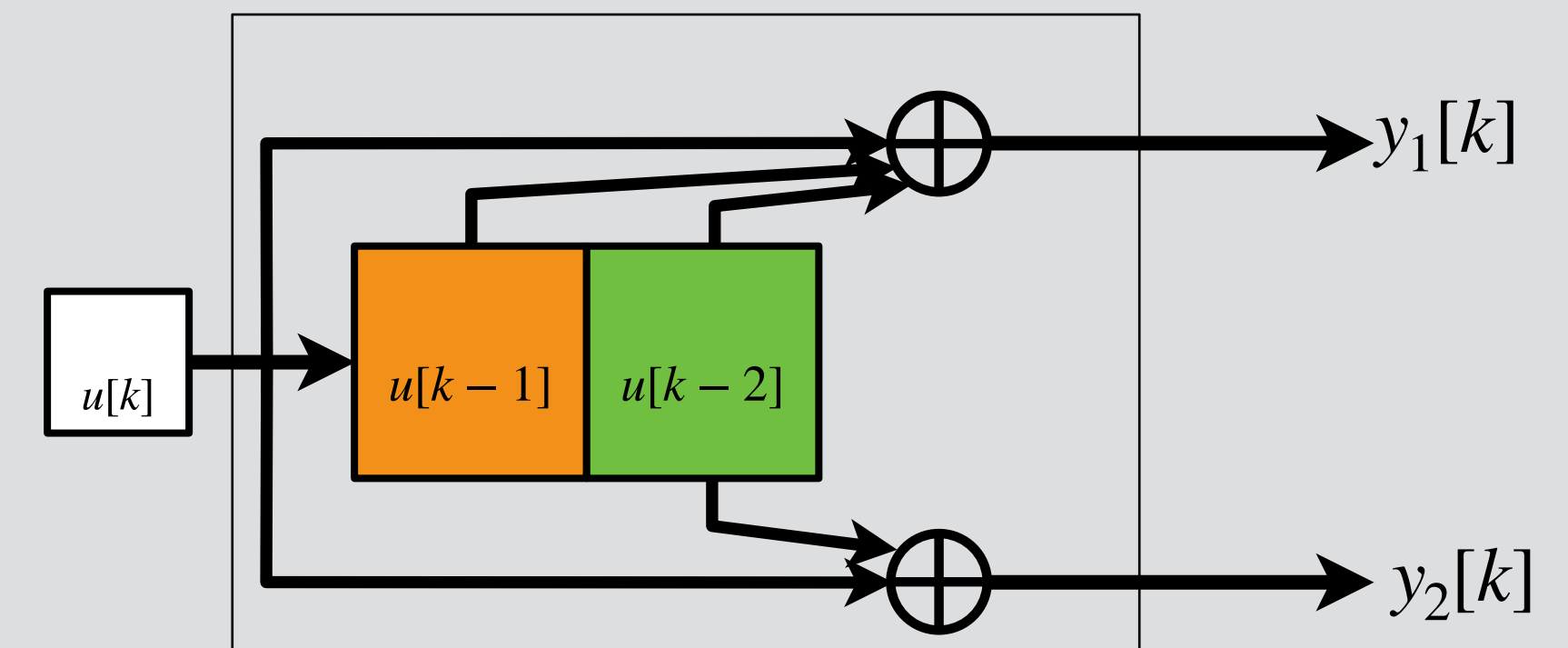


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Corrected: $11101\mathbf{1}1101$

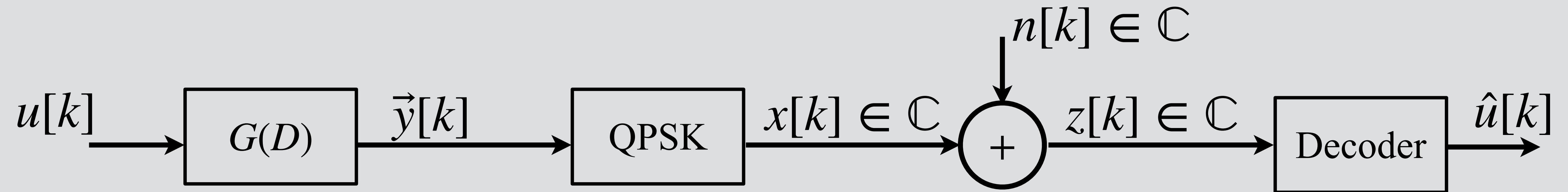
Decoded: $\hat{u} = 10011$



Backtrack to find optimal path!

Soft Decoding

Consider an AWGN channel with the following encoder:



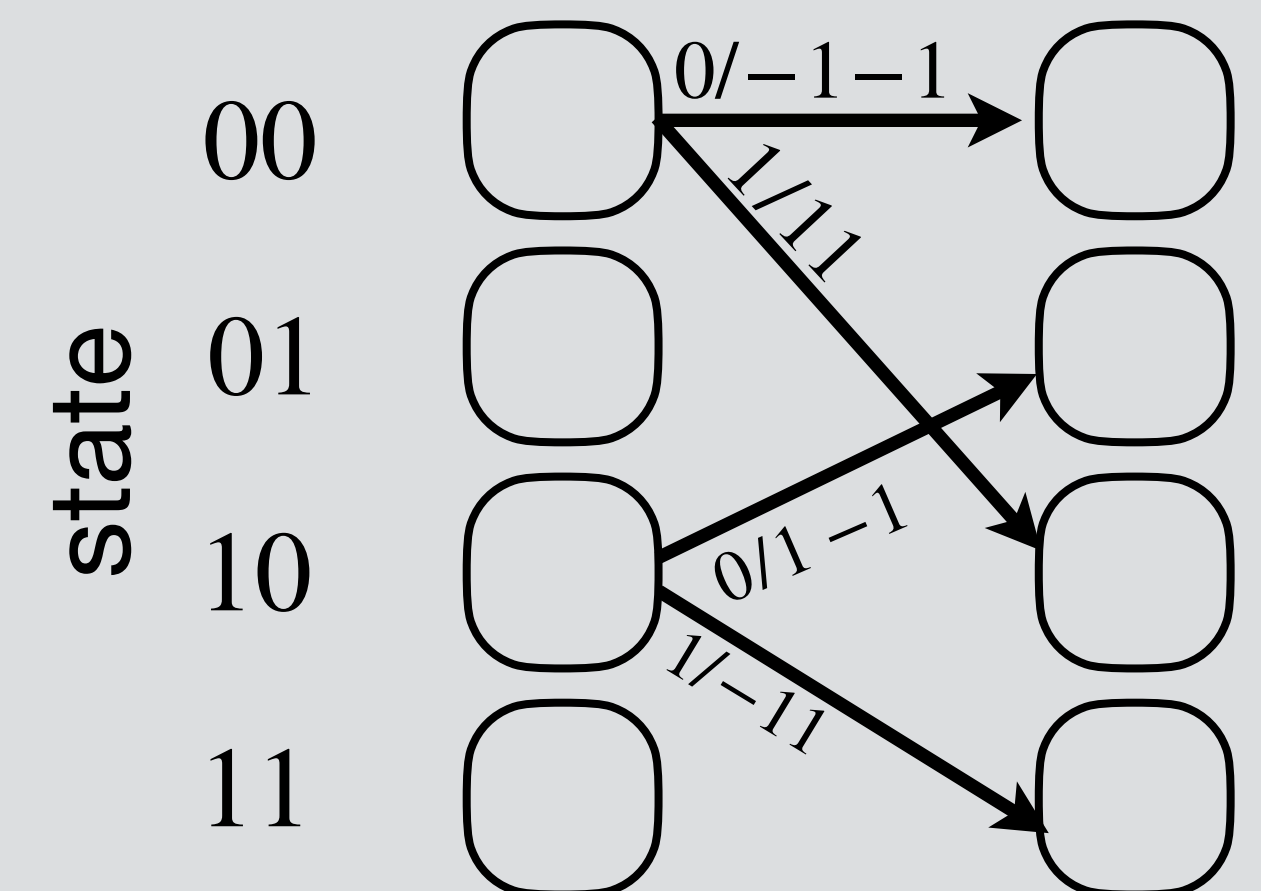
Define *branch metric* $\lambda_k(s[k] \rightarrow s[k + 1])$

Cost of moving from one state to another:

$$\lambda_k(s[k] \rightarrow s[k + 1]) = \|z[k] - x[k]\|^2$$

Without noise, cost = 0!

$$\hat{z}[k] = -1.1 + 0.8j$$

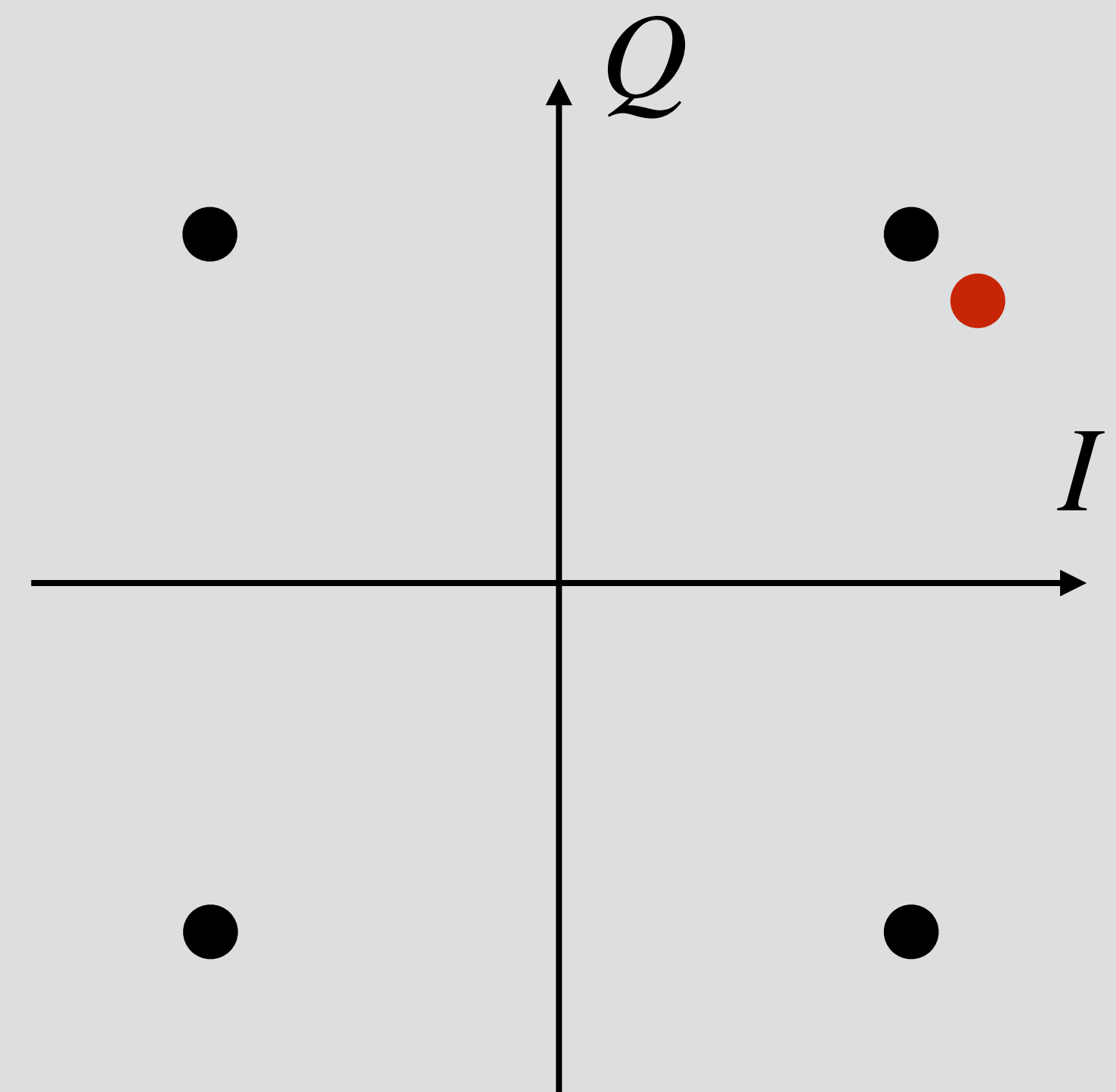
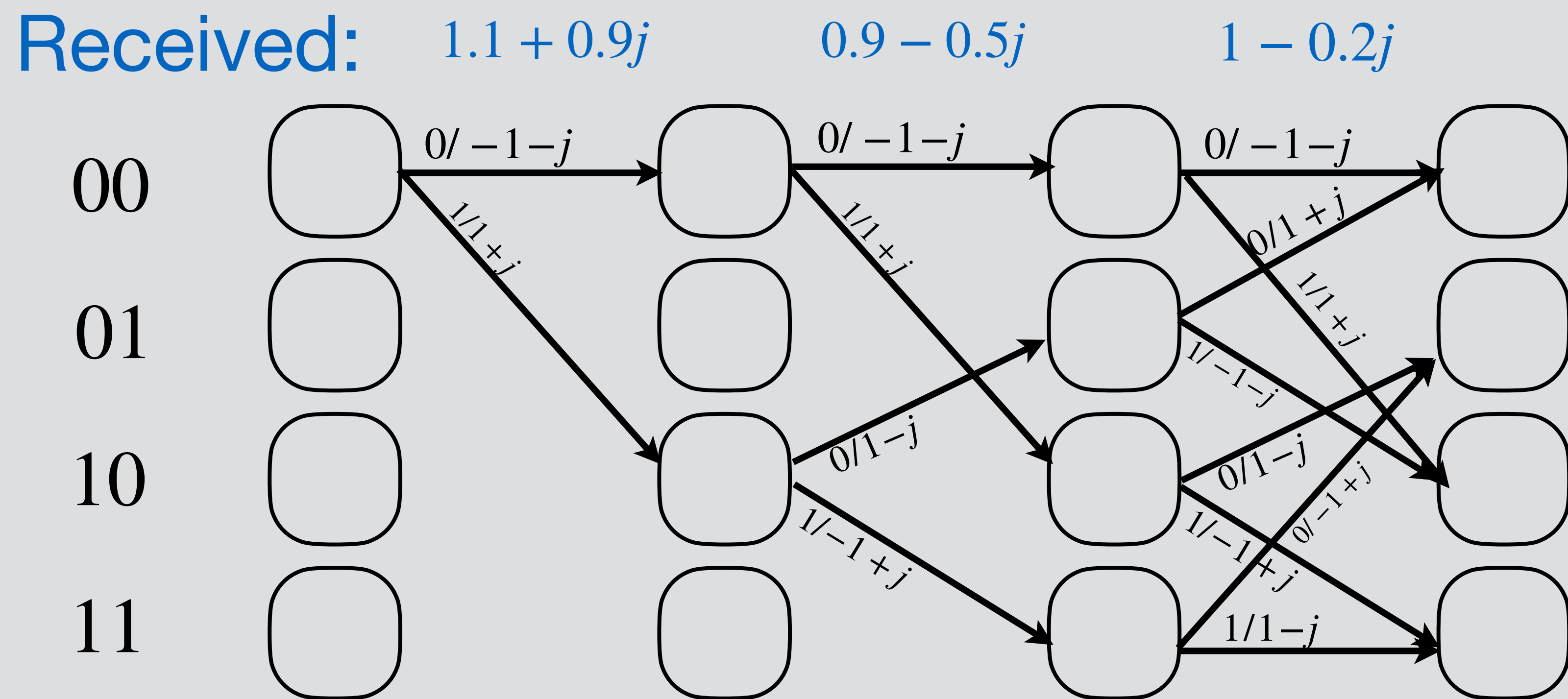
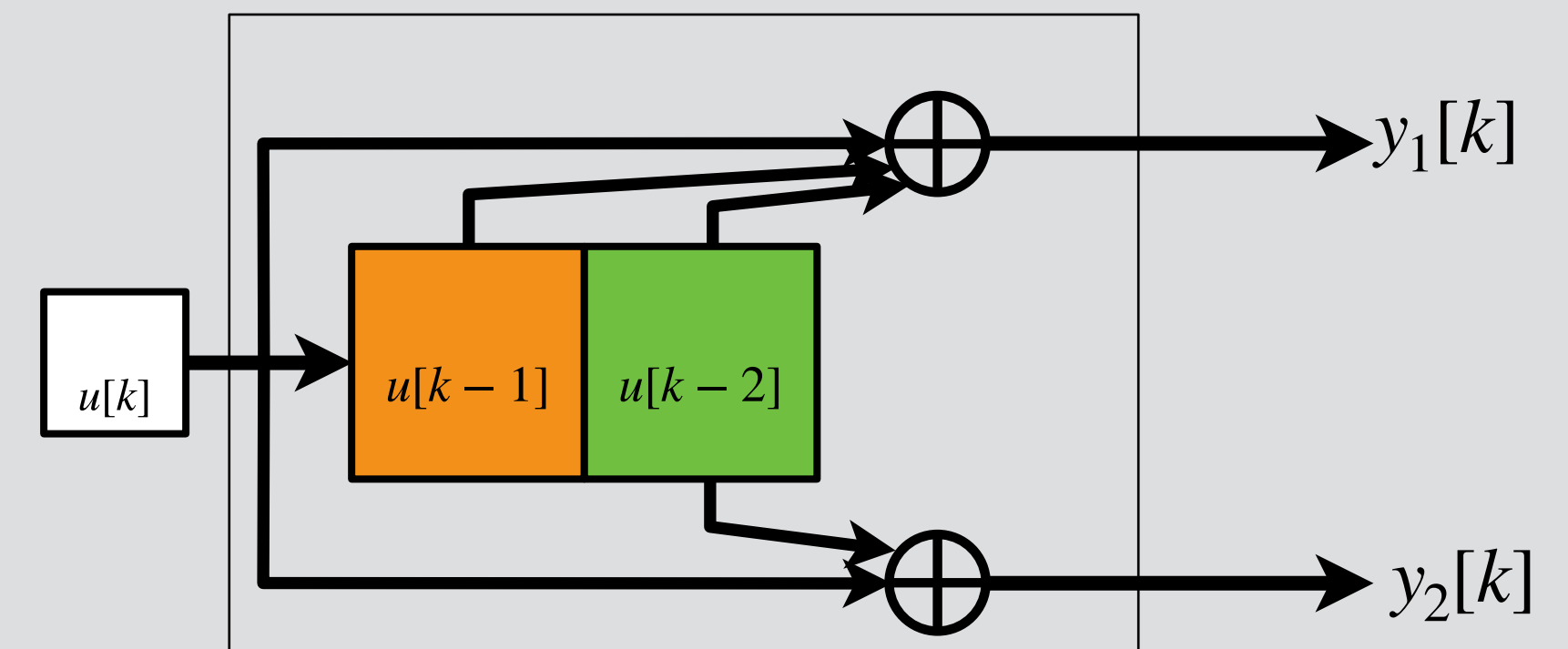


Trellis diagram and Viterbi Algorithm

Sent: $u = 100$, Code: $y = 111011$

Baseband: $x = [1 + 1j, 1 - 1j, 1 + 1j]$

Received: $z = [1.1 + 0.9j, 0.9 - 0.5j, 1 - 0.2j]$

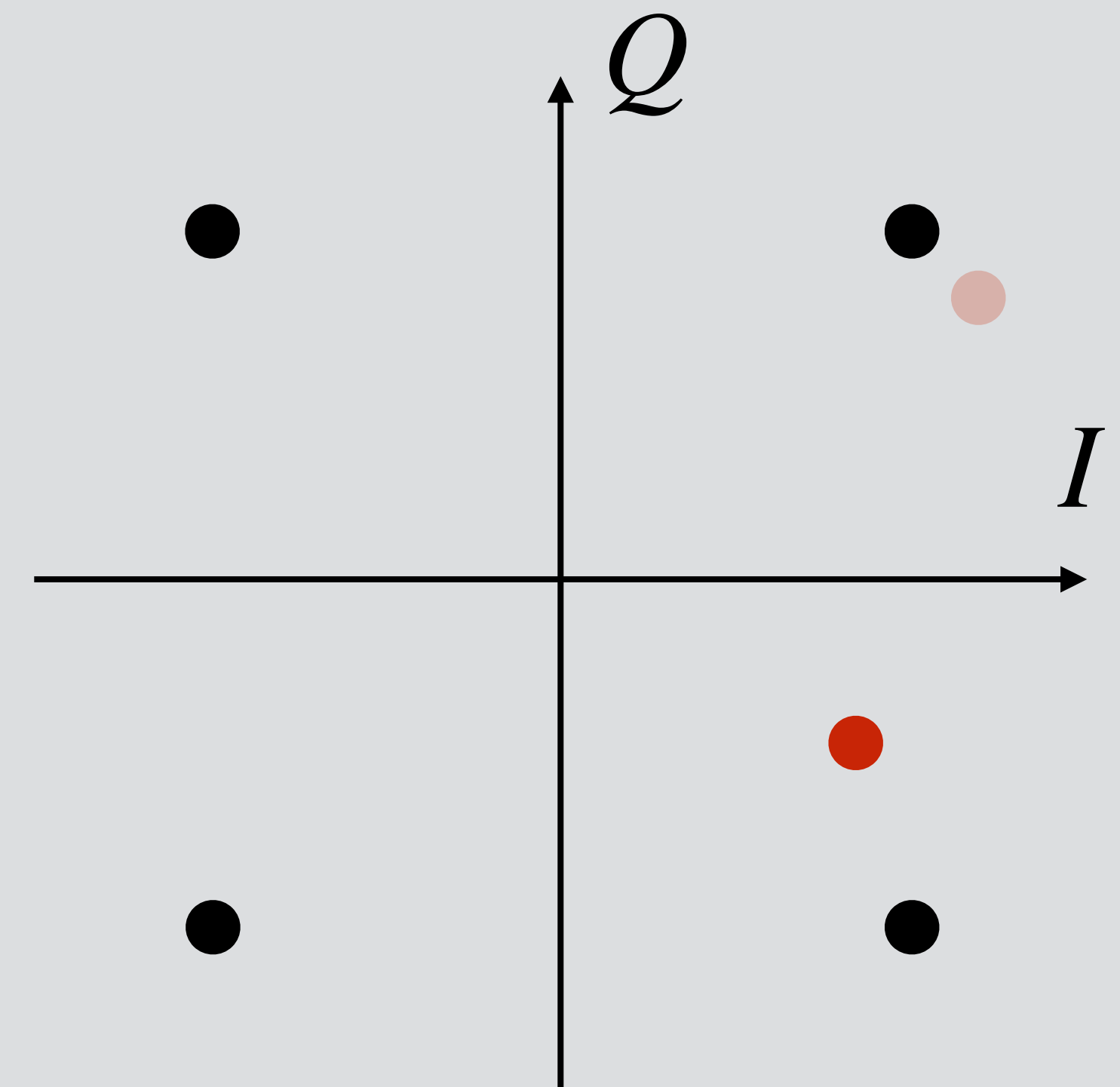
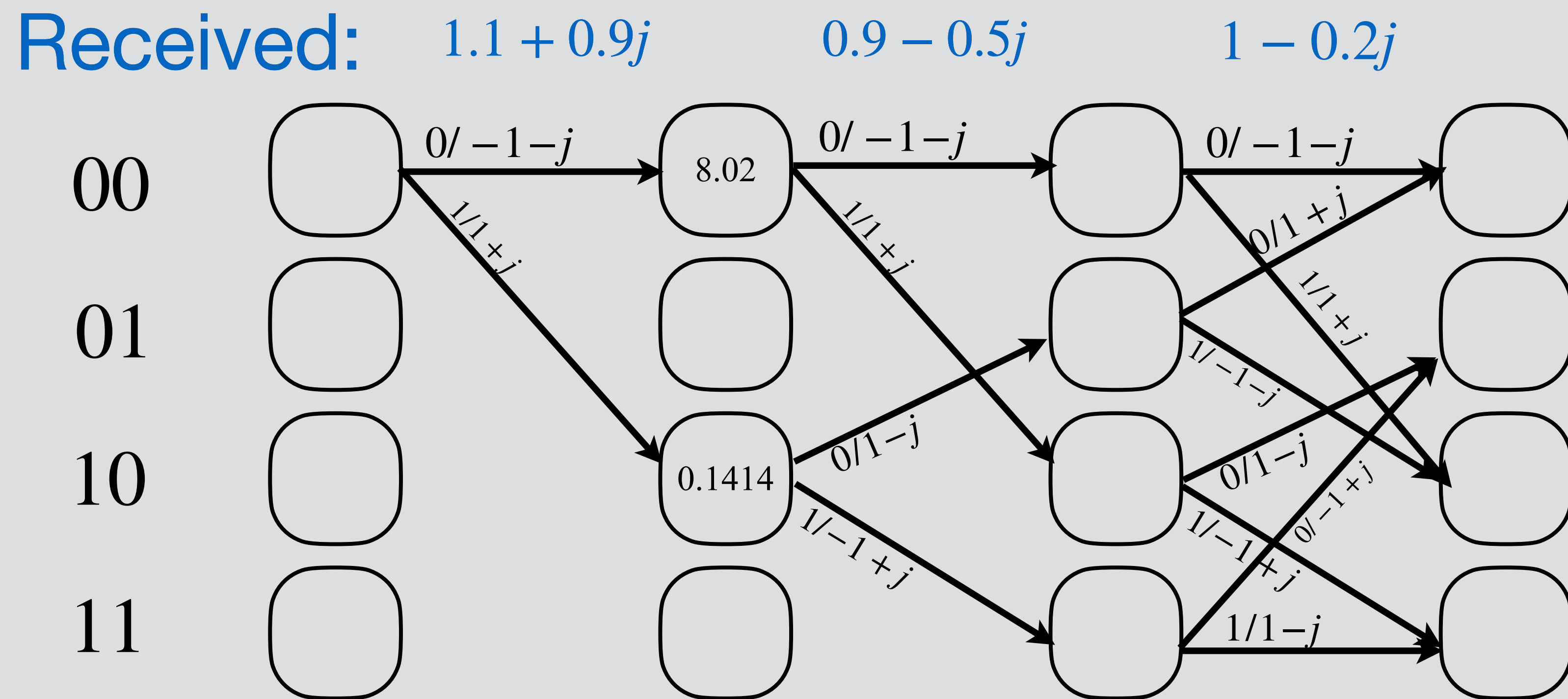
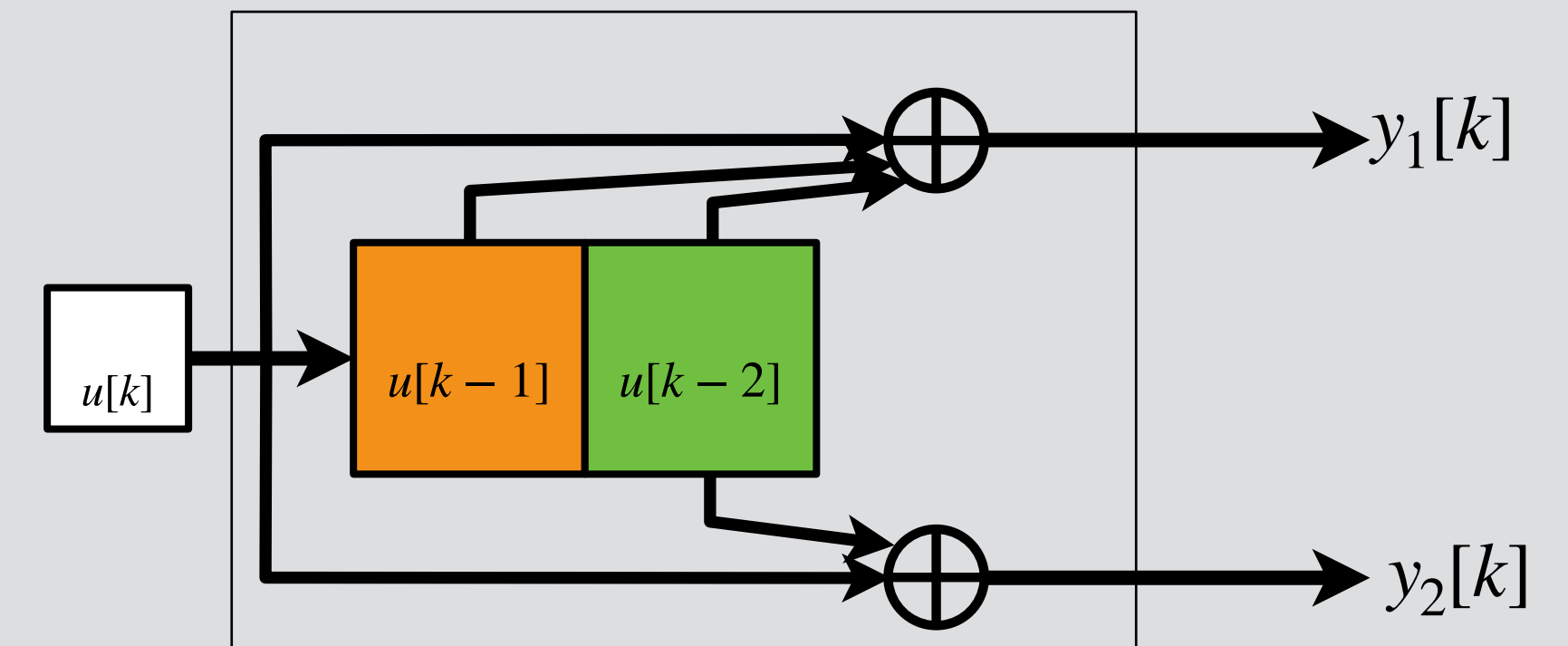


Trellis diagram and Viterbi Algorithm

Sent: $u = 100$, Code: $y = 111011$

Baseband: $x = [1 + 1j, 1 - 1j, 1 + 1j]$

Received: $z = [1.1 + 0.9j, 0.9 - 0.5j, 1 - 0.2j]$

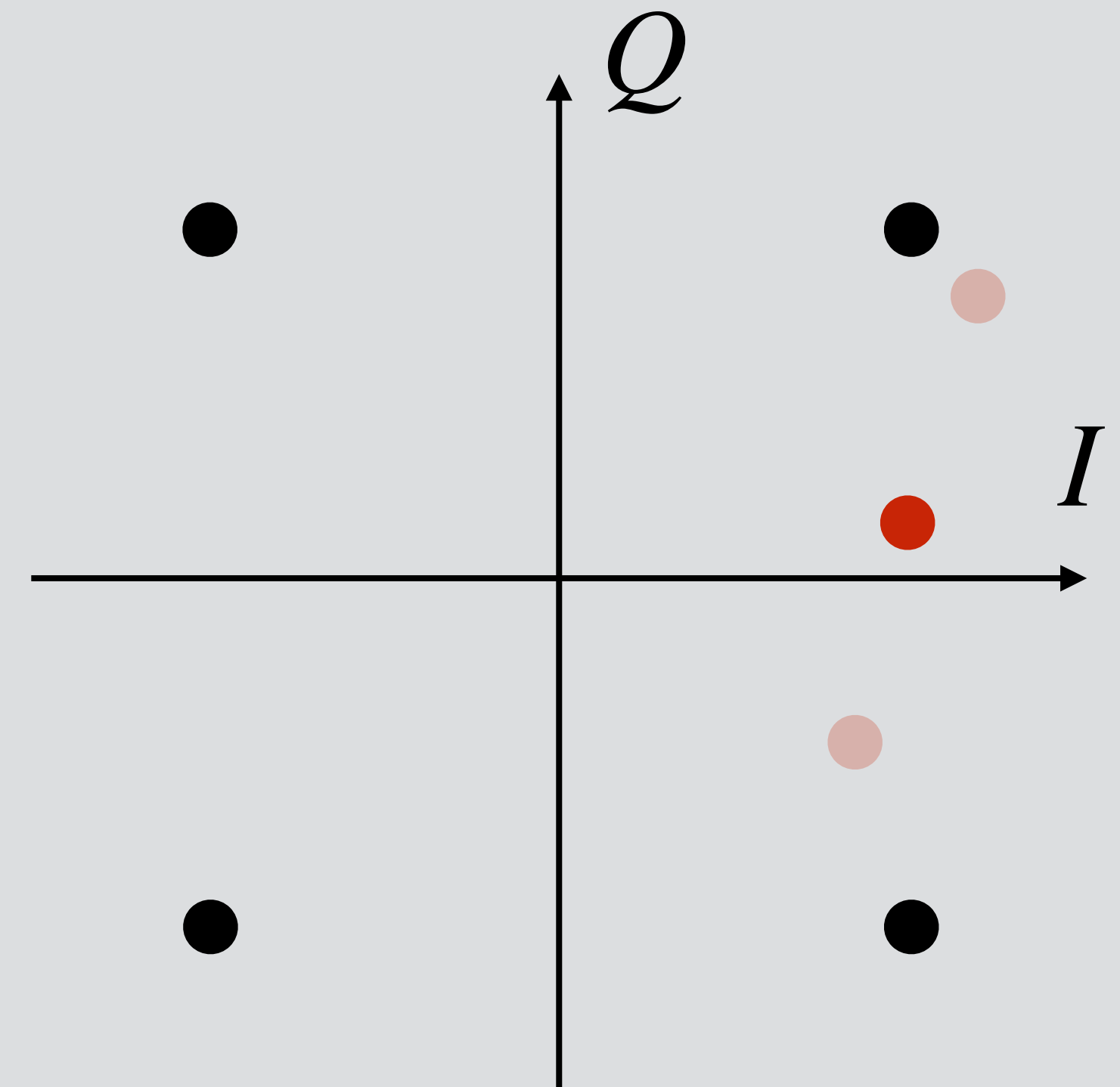
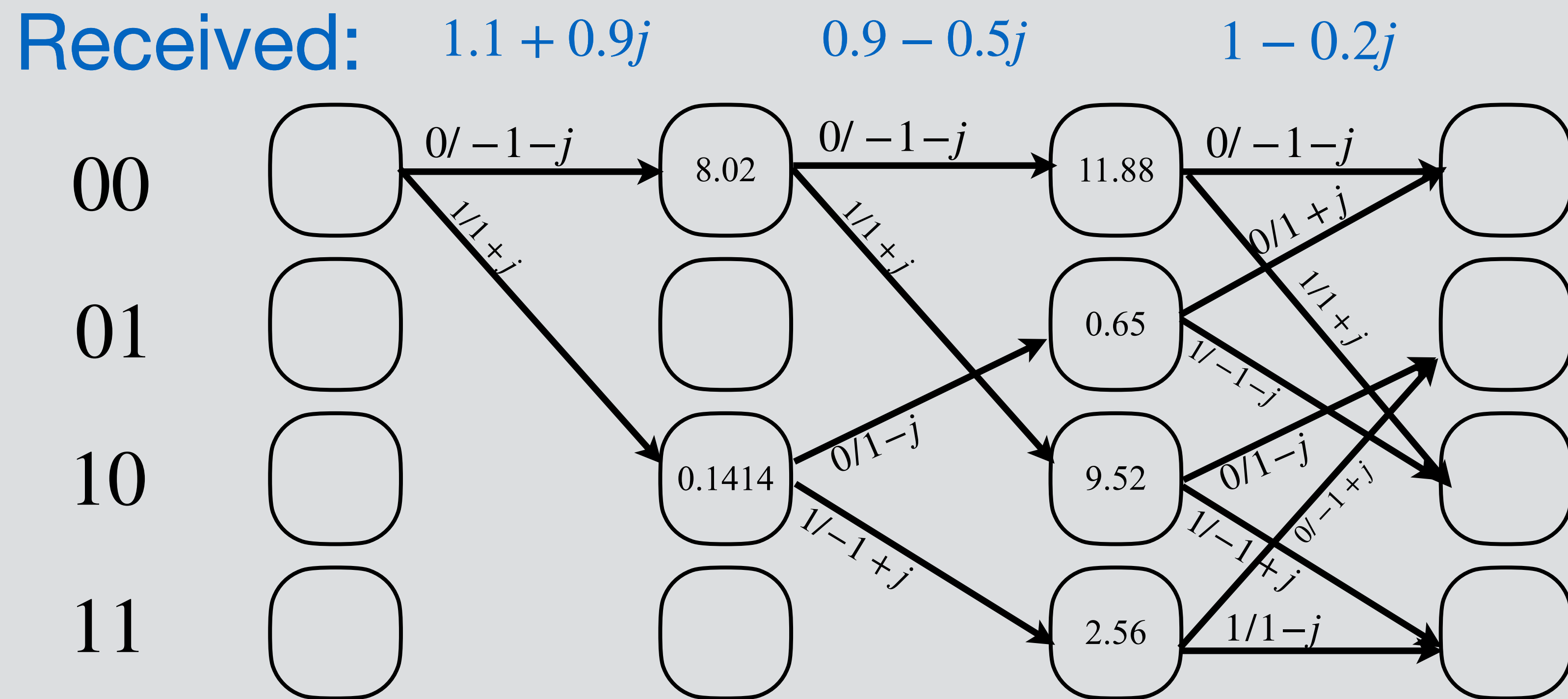
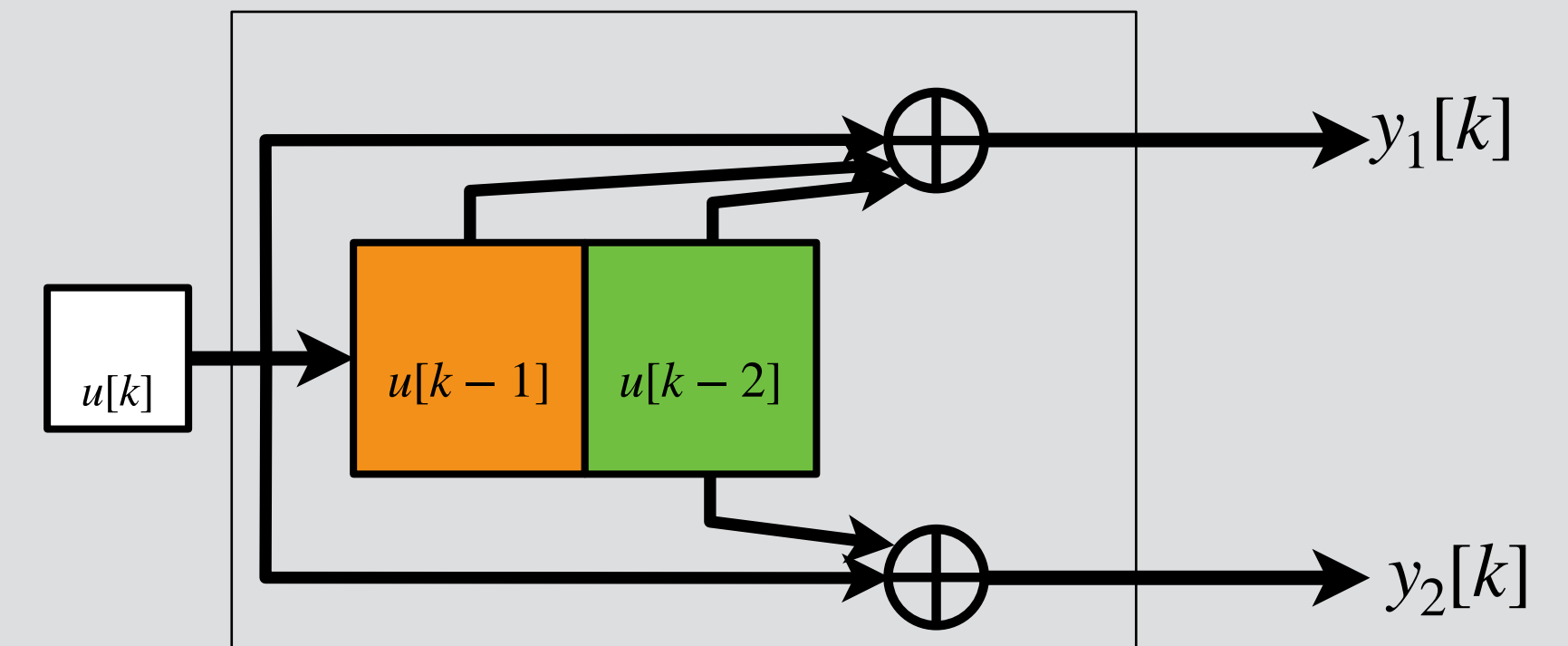


Trellis diagram and Viterbi Algorithm

Sent: $u = 100$, Code: $y = 111011$

Baseband: $x = [1 + 1j, 1 - 1j, 1 + 1j]$

Received: $z = [1.1 + 0.9j, 0.9 - 0.5j, 1 - 0.2j]$

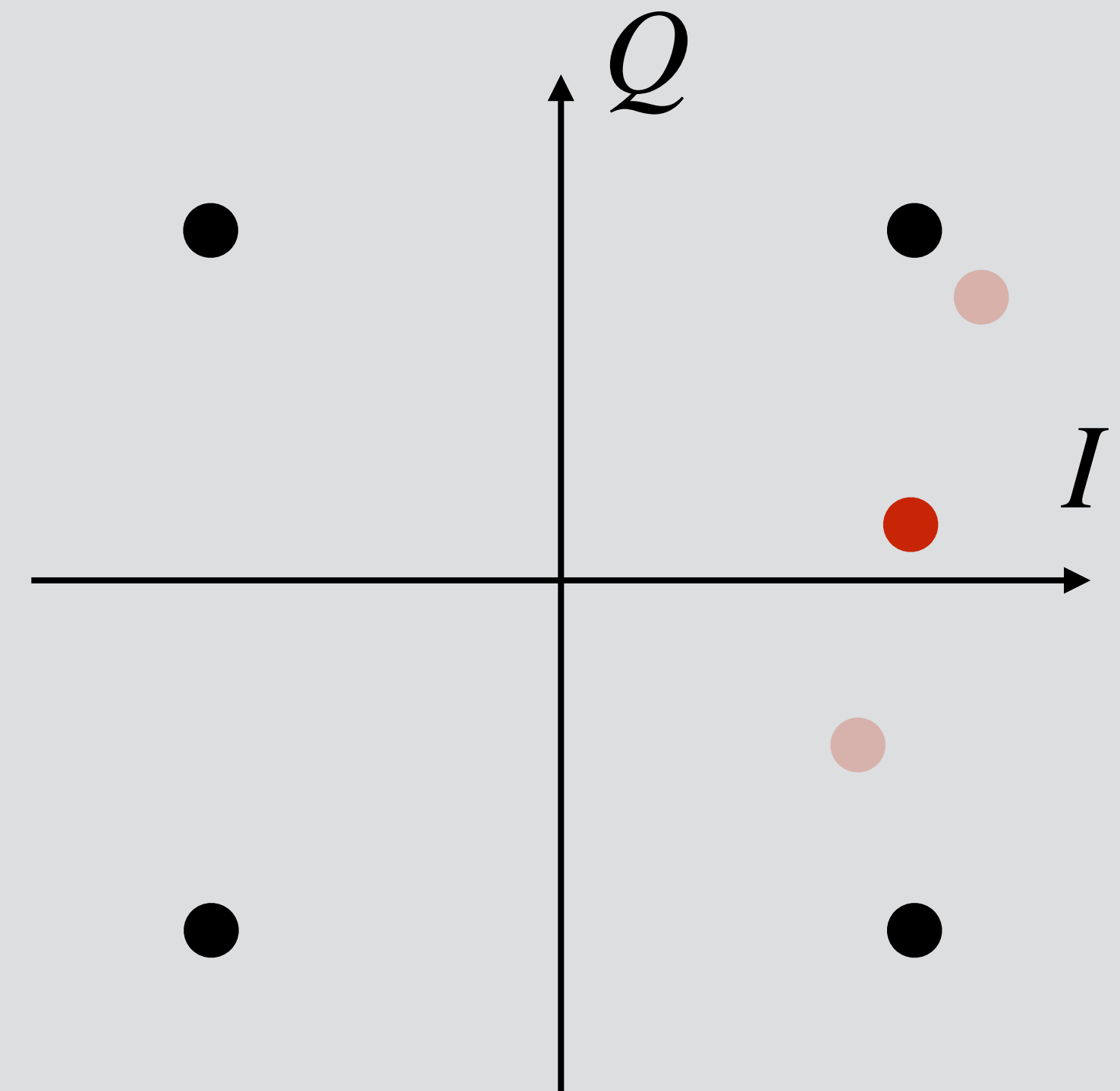
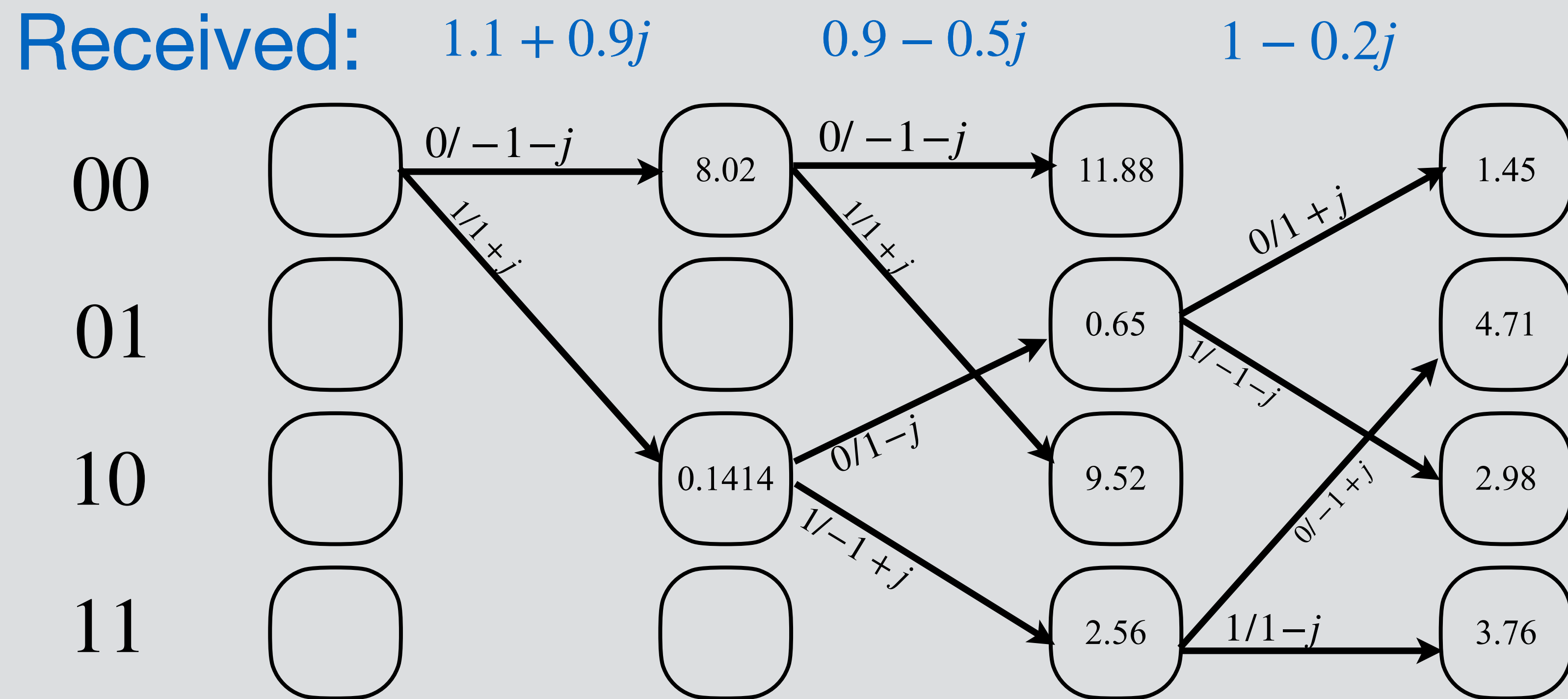
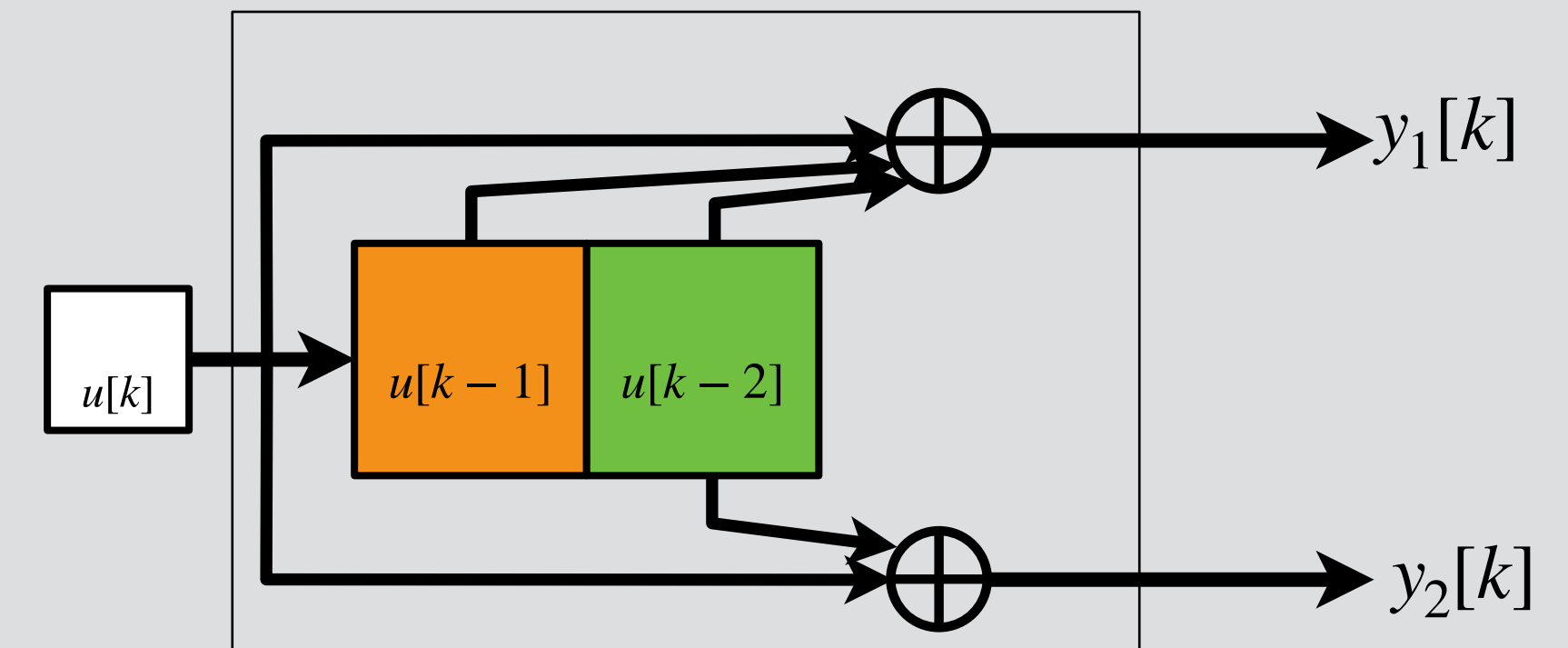


Trellis diagram and Viterbi Algorithm

Sent: $u = 100$, Code: $y = 111011$

Baseband: $x = [1 + 1j, 1 - 1j, 1 + 1j]$

Received: $z = [1.1 + 0.9j, 0.9 - 0.5j, 1 - 0.2j]$

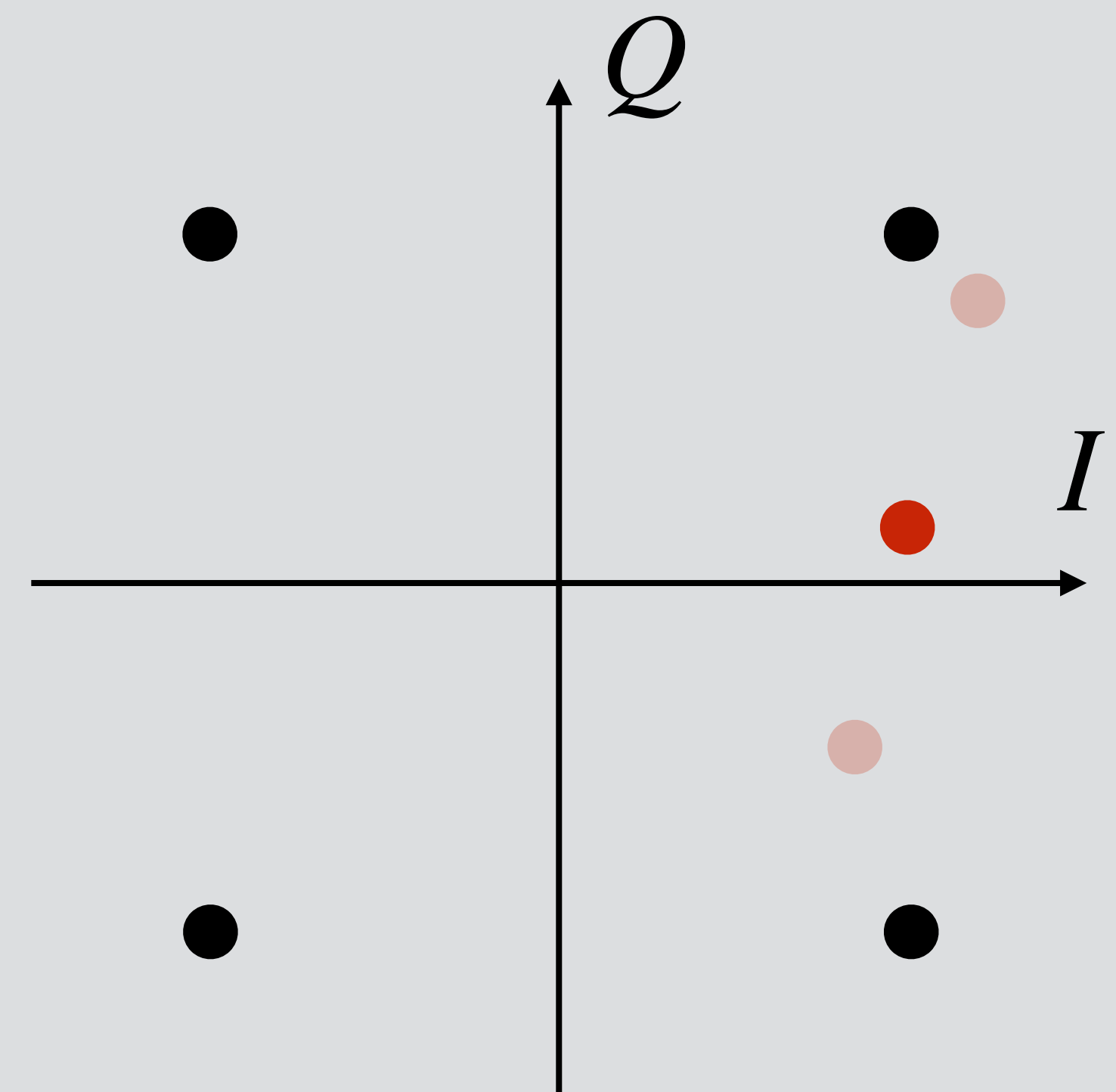
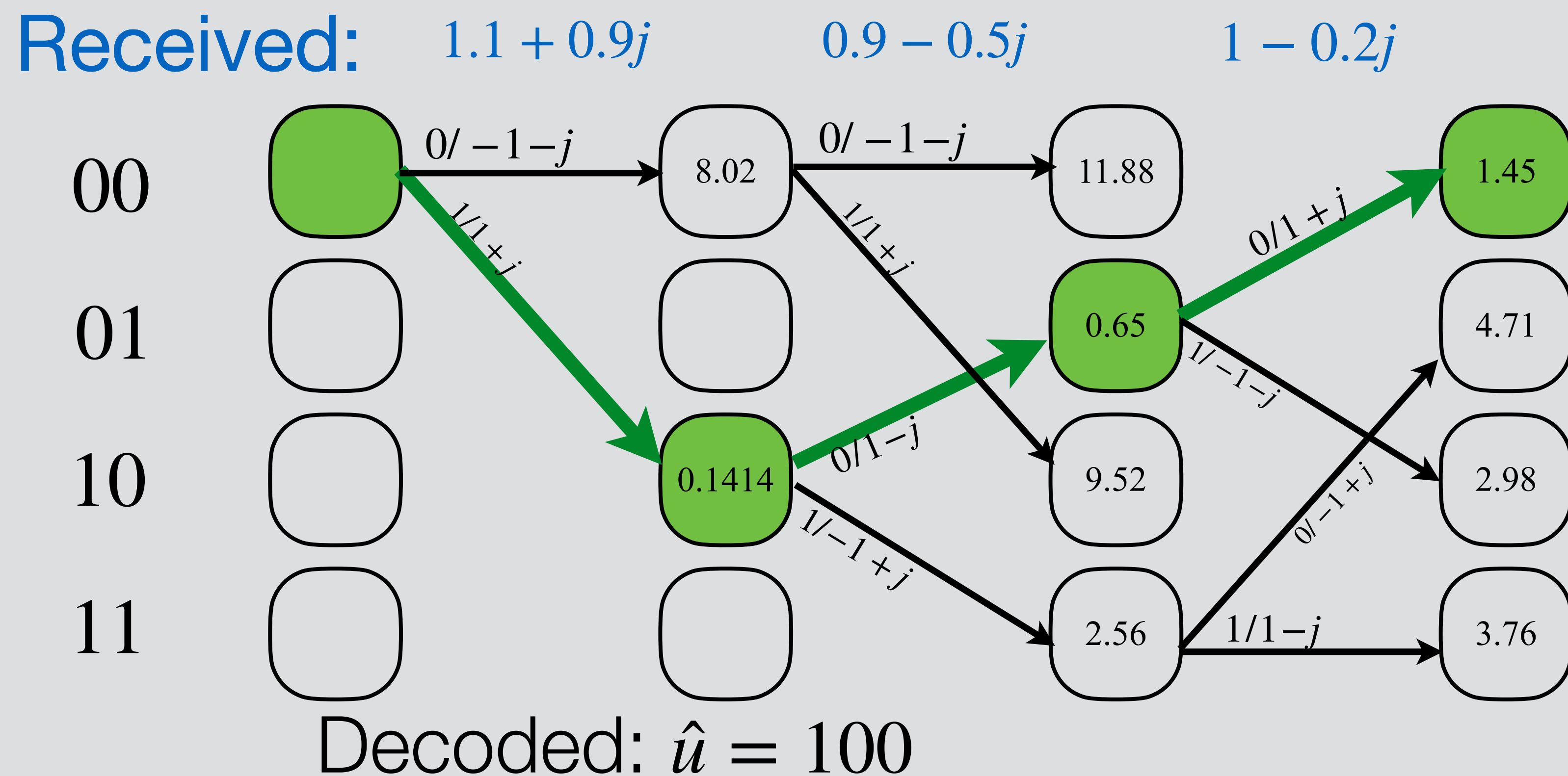
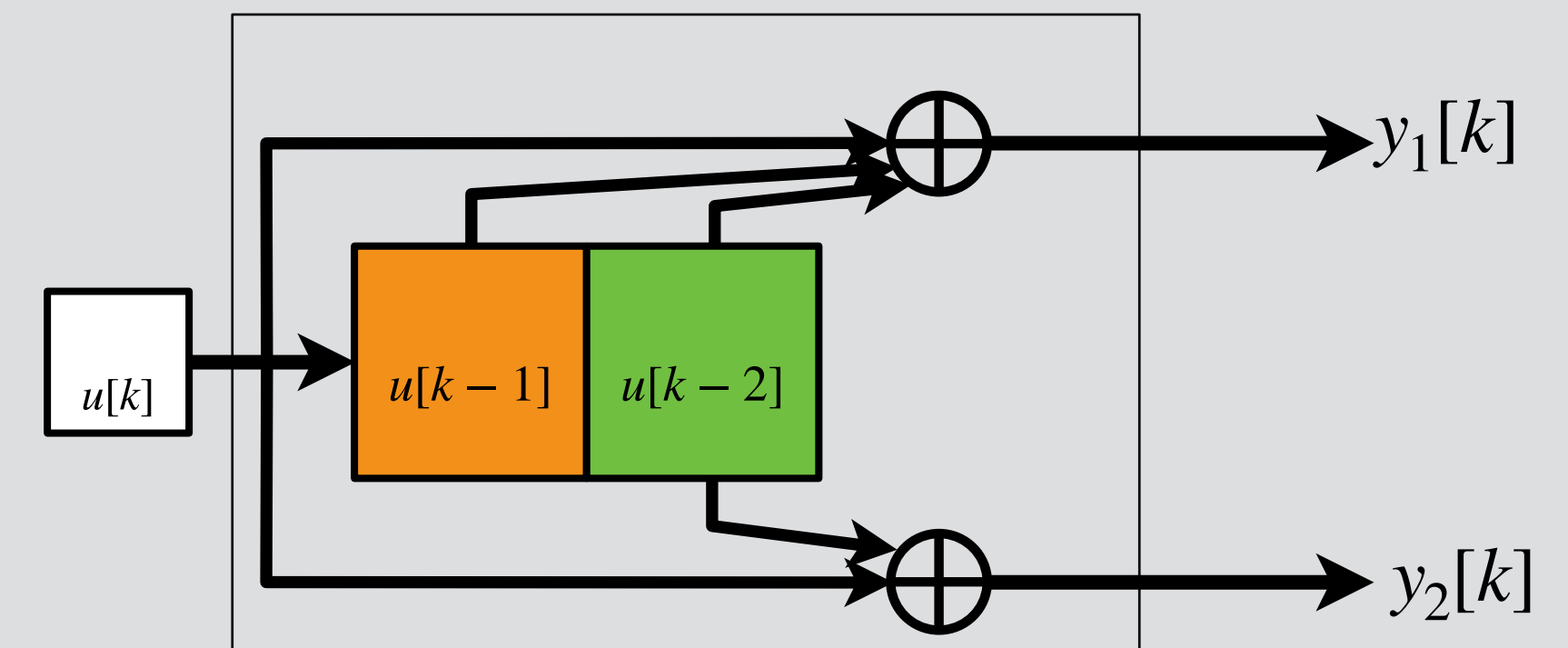


Trellis diagram and Viterbi Algorithm

Sent: $u = 100$, Code: $y = 111011$

Baseband: $x = [1 + 1j, 1 - 1j, 1 + 1j]$

Received: $z = [1.1 + 0.9j, 0.9 - 0.5j, 1 - 0.2j]$



Viterbi

- Start with all zeros state (encoder, decoder)
- Compute costs for every branch
- Store cumulative costs in each state
 - Choose the minimum cumulative cost if more than one
 - Eliminate impossible branch
- Repeat for all bits
- Choose cumulative lowest cost path. Retract back to determine bits

Performance comparison

- In general, careful analysis is complex and challenging.
- Instead will provide some metric for comparison
- Metrics:
 - d_{free} minimum Hamming distance (number of bits which will result in error — similarly to d_{min})
 - $\gamma_f = rd_{\text{free}}$ coding gain - improvement in “SNR” between coded and uncoded
 - N_e - number of neighboring (minimum distance) error events at d_{free}
 - N_1 - number of error events at $d_{\text{free}} + 1$
 - N_2 - number of error events at $d_{\text{free}} + 2$

Rate 1/2 Maximum Free Distance Codes

2^v	$g_{11}(D)$	$g_{12}(D)$	d_{free}	γ_f	γ_f (dB)	N_e	N_1	N_2	Example Uses
4	7	5	5	2.5	3.98	1	2	4	
8	17	15	6	3	4.77	1	3	5	[13 15] systematic FB used in LTE & WCDMA turbo code,
16	31	27	7	3.5	5.44	2	3	4	
16	23	33	7	3.5	5.44	2	4	6	GSM
32	77	45	8	4	6.02	2	3	8	IS-54 TDMA
64	147	135	10	5	6.99	12	0	53	
64	133	171	10	5	6.99	11	0	38	802.11a/g, NASA
64	155	137	9	4.5	6.53	1	6	11	802.11b
128	313	275	10	5	6.99	1	6	13	
256	751	557	12	6	7.78	10	9	30	
256	753	561	12	6	7.78	11	0	50	IS-95 CDMA
512	1755	1363	12	6	7.78	1	8	8	
1024	3645	2671	14	7	8.45	19	0	80	
2048	6643	5175	14	7	8.45	1	10	25	
4096	14677	11651	15	7.5	8.75	2	10	29	
8192	22555	37457	16	8	9.00	5	15	21	

Higher Order Codes with Puncturing

- Use a baseline $r=1/2$ code
- Discard certain outputs
- Specified by a puncturing matrix

- $r = \frac{2}{3}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- $r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$

- $r = \frac{5}{6}, P = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$

Puncturing Example

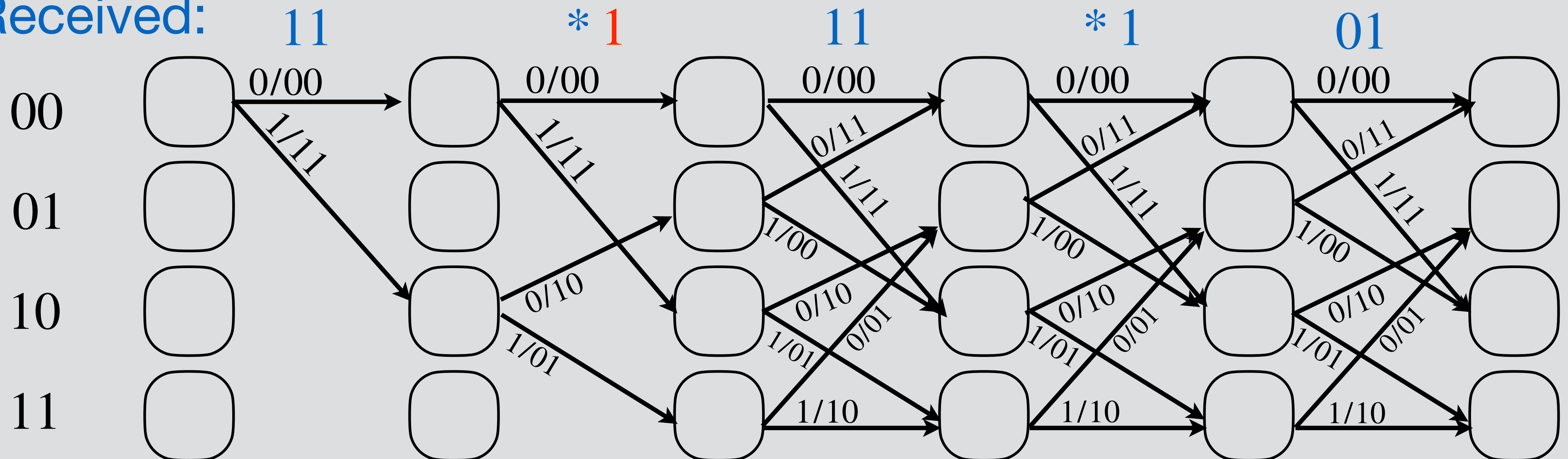
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

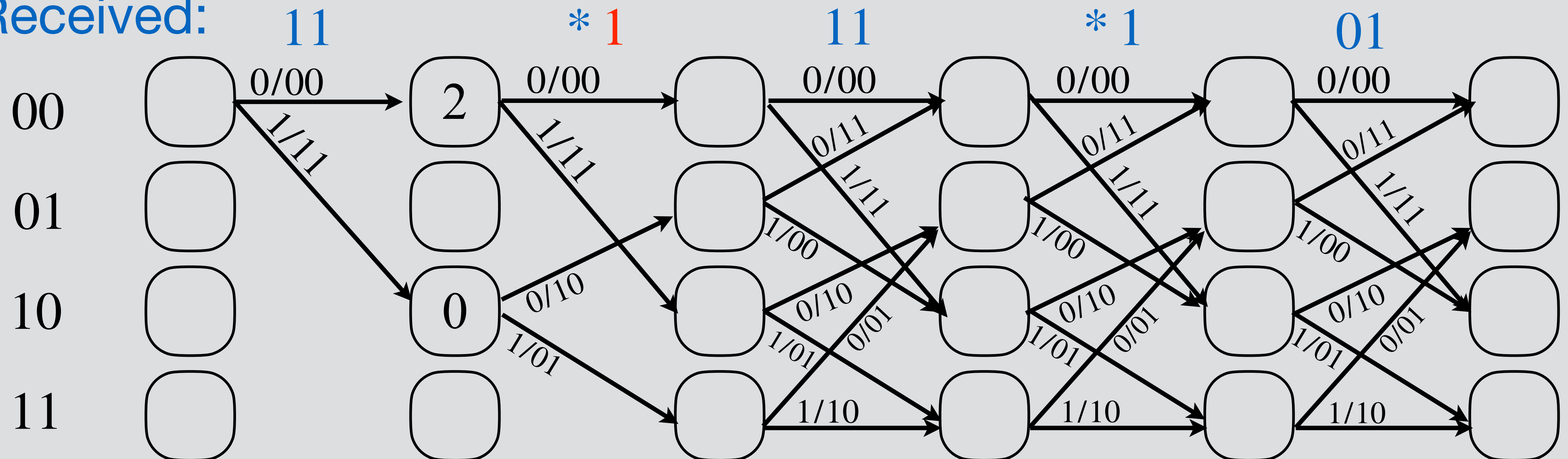
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

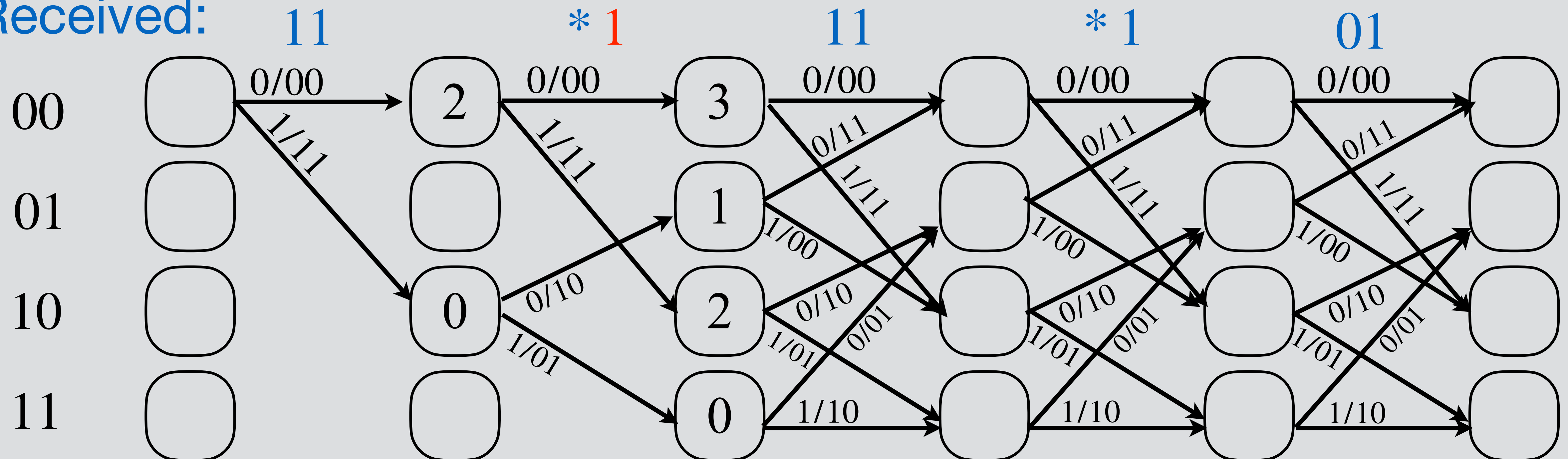
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

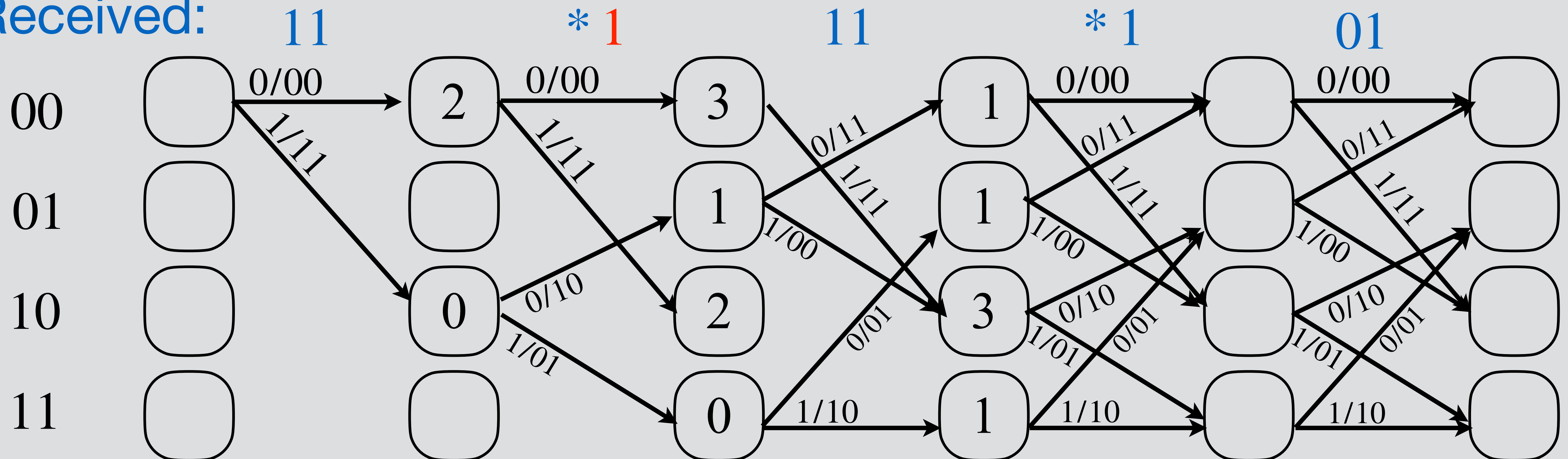
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

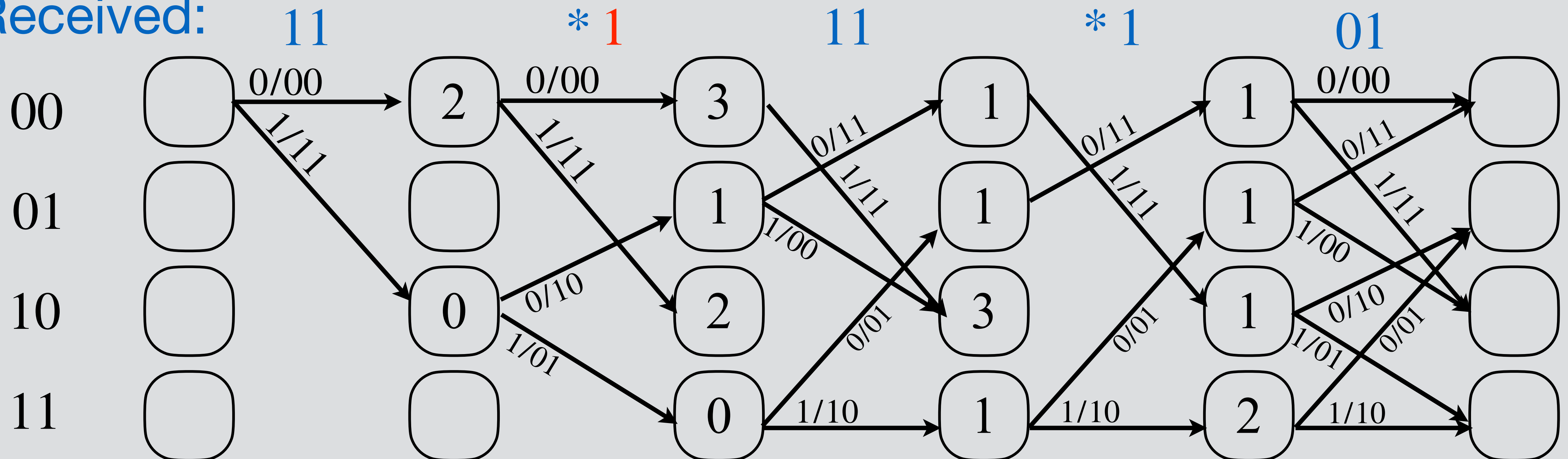
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

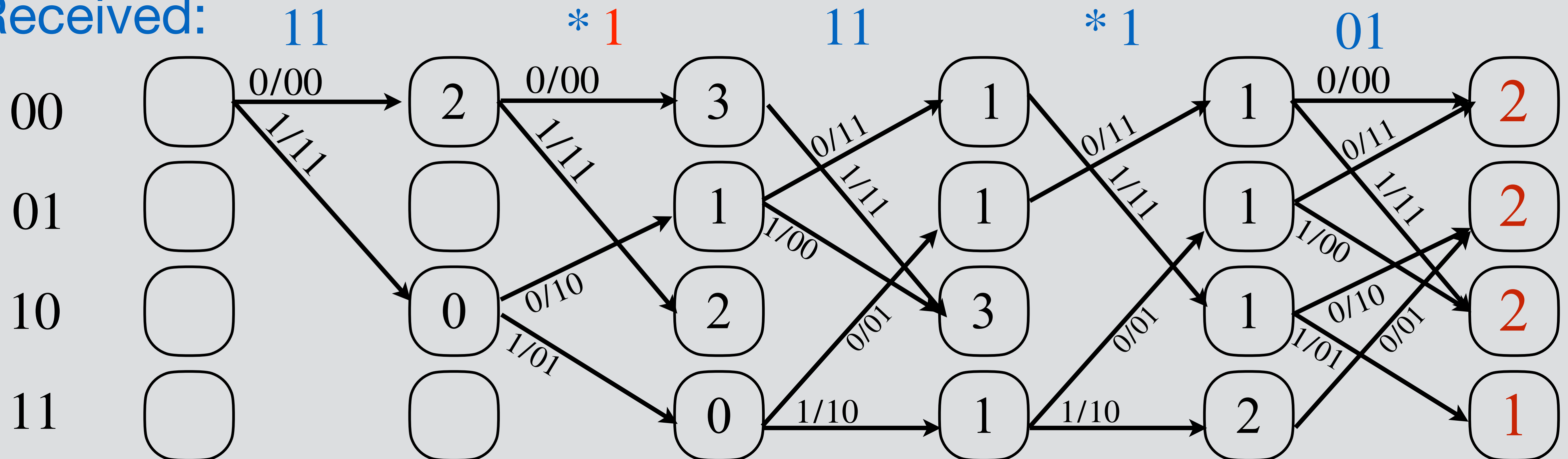
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:



Puncturing Example

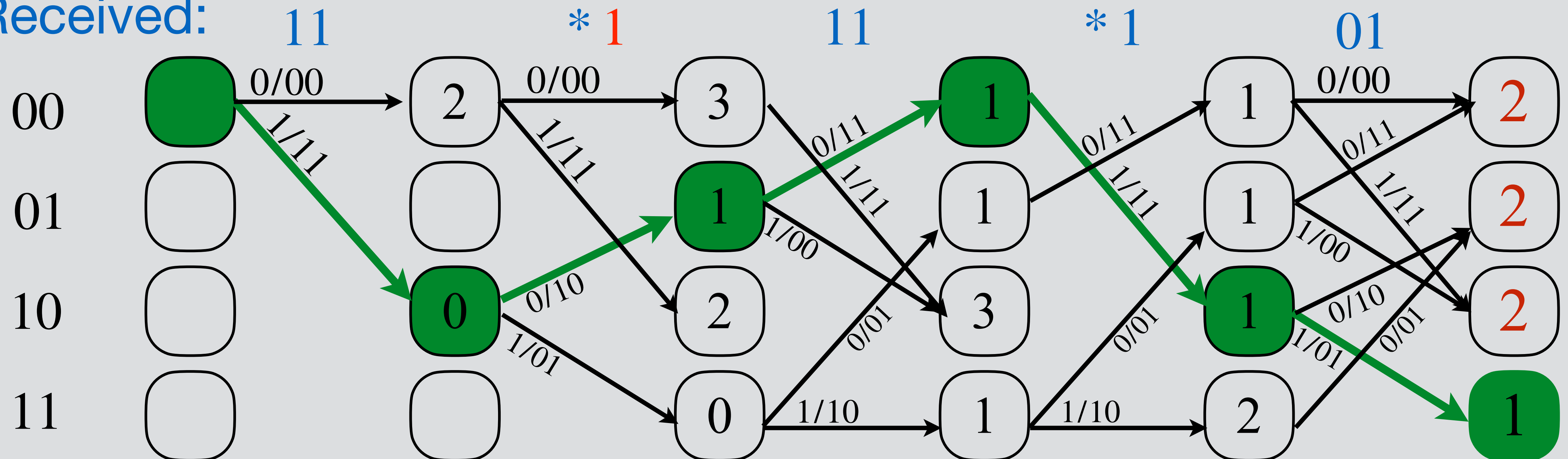
Send: $u = 10011$

$$r = \frac{3}{4}, P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Code: $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Punctured: $\begin{bmatrix} 1 & * & 1 & * & 0 \\ 1 & 0 & 1 & 1 & 1 \end{bmatrix}$

Received:

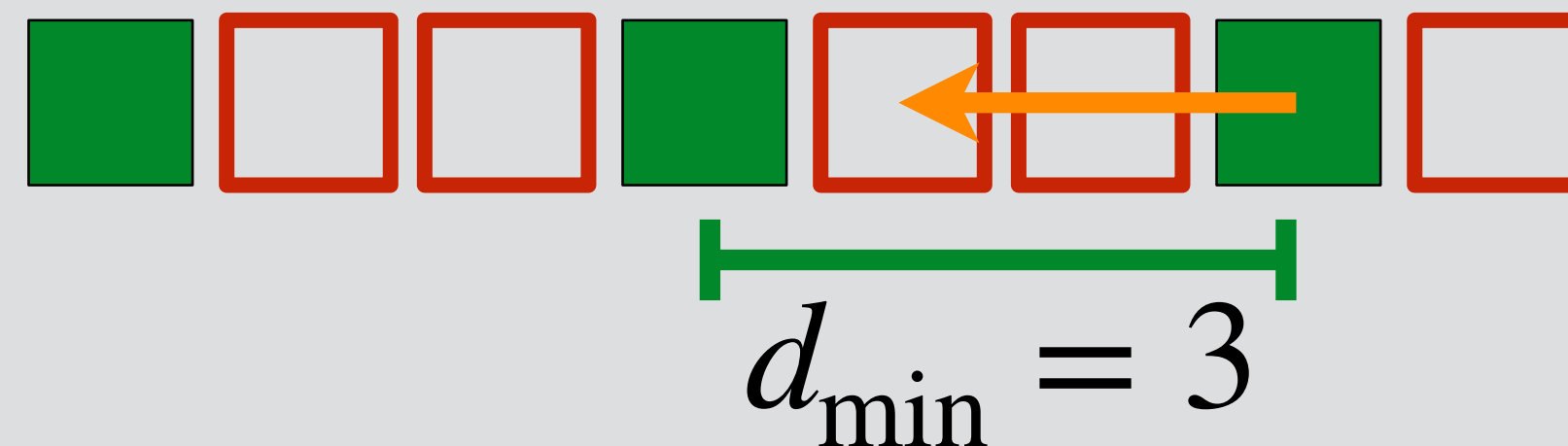


Block Codes

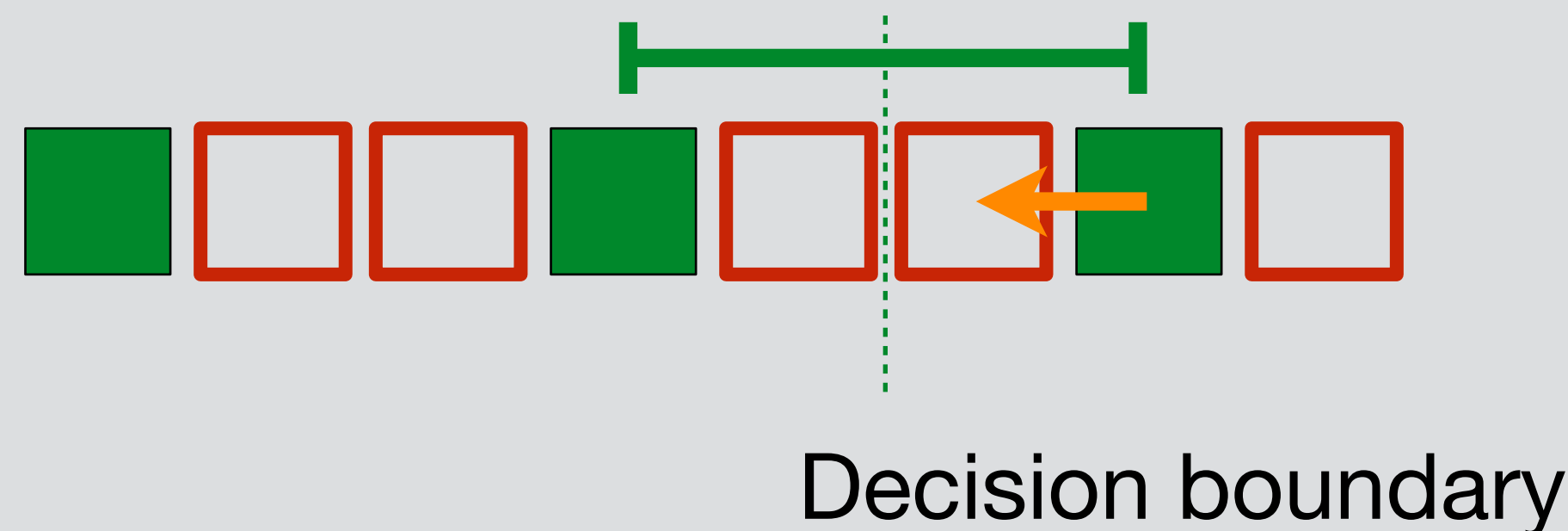
- Operate on block

Hamming distance

- If hamming Distance between codewords is $\geq d_{\min}$
 - We can detect at most $d_{\min} - 1$ errors



- We can correct at most $(d_{\min} - 1)/2$ errors



Parity Check

$$c_1 \oplus c_2 \oplus \cdots \oplus c_N \oplus c_p = 0$$

Q: What is $d_{\min}$?

A: Change one bit, changes parity bit, so $d_{\min} = 2$

Detects 1 bit, corrects 0

2D Parity

- Break up data into multiple rows
 - Append parity on across each row
 - Append parity down each column
 - Add r , parity of row parity
- Example, rows/columns k_2/k_1
 - Row parity bits: k_2
 - Column parity bits k_1
 - Total bits: $(k_1 + 1)(k_2 + 1)$
 - Rate: $\frac{k_1 k_2}{(k_1 + 1)(k_2 + 1)}$

Data	Row parity
Column parity	r

2D Parity

- Single error flips row and column parities. Can be corrected!
- Double can be detected

Data	Row parity
Column parity	<i>r</i>

Block Codes: Hamming Codes

- Come up with a code, in which each parity equation reveals a bit on the error location
- Basic idea:
 - Send a block size $n = 2^m - 1$
 - m bits are used for parity, the rest for data
 - Each parity bit is computed on half the block

Hamming Code Construction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇

- Each column is binary representation of that index

Hamming Code Construction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
	1	0	1	0	1	0	1
	0	1	1	0	0	1	1
	0	0	0	1	1	1	1

- Each column is binary representation of that index
- Any index with single bit, is used for parity

Hamming Code Construction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
	p ₁	p ₂		p ₄			
	1	0	1	0	1	0	1
	0	1	1	0	0	1	1
	0	0	0	1	1	1	1

- Each column is binary representation of that index
- Any index with single bit, is used for parity
- All others sequentially filled with input bits

Hamming Code Construction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
	p ₁	p ₂	d ₁	p ₄	d ₂	d ₃	d ₄
	1	0	1	0	1	0	1
	0	1	1	0	0	1	1
	0	0	0	1	1	1	1

- Each column is binary representation of that index
- Any index with single bit, is used for parity
- All others sequentially filled with input bits
- Parity is computed for all indexes with the same index bit

Hamming Code Construction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
	p ₁	p ₂	d ₁	p ₄	d ₂	d ₃	d ₄
p ₁	1	0	1	0	1	0	1
p ₂	0	1	1	0	0	1	1
p ₄	0	0	0	1	1	1	1

- Each column is binary representation of that index
- Any index with single bit, is used for parity
- All others sequentially filled with input bits
- Parity is computed for all indexes with the same index bit

Hamming Error Correction

Example: $2^n - 1 = 7$, This is a (7,4) code

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
	p ₁	p ₂	d ₁	p ₄	d ₂	d ₃	d ₄
p ₁	1	0	1	0	1	0	1
p ₂	0	1	1	0	0	1	1
p ₄	0	0	0	1	1	1	1

- Error is in binary $p_4p_2p_1$ position!
- Example:
 - 001, error in p_1
 - 111, error in d_4

Summary

- A single parity-check detects single errors
- Hamming codes use m equations to correct one error in $2^m - 1$ bits
 - Useful if the error rate is low
 - ECC RAM uses hamming codes!
 - More complicated algorithms are needed when multiple errors occur